

ALUCLU: A Bounded Multi-Lane Streaming Memory Architecture

Adaptive Layered Unified Context with Learned Updates

Kaan Kadir Aluclu

Independent Researcher and Software Engineer

github.com/kaannsaydamm/ALUCLU

Version 0.1.0 – 30 July 2026

Abstract

ALUCLU is a decoder-only research architecture for causal sequence modeling that places four physically different, explicitly bounded memory lanes inside a single residual block: a dense recurrent fast-weight lane based on a Gated Delta Rule-2 update, exact sliding-window softmax attention, a fixed-capacity residual-surprise exact key-value cache, and a segmented episodic lane with exact recent factors and low-rank archived capsules. All active branches read the same normalized token representation and are combined by learned token-wise mixture coefficients before a SwiGLU feed-forward stage. The design is motivated by a simple systems observation: recency, dense statistical continuity, sparse exceptional events, and compressed temporal episodes impose different storage and access requirements, so forcing all four into one state representation creates avoidable trade-offs.

This paper specifies the implemented recurrence, cache replacement policy, episodic life cycle, mass-conserving router, thin-QR small-core SVD compression, persistent-state accounting, and causal execution semantics. It also states what the current evidence does and does not establish. The reference implementation is a portable token-recurrent correctness backend, not a fused accelerator kernel and not a trained frontier model. Fifty automated tests validate causality, scan/step equivalence in evaluation mode, bounded state, compression error accounting, priority retention, routing mass conservation, and finite gradients on small cases. A deliberately small CPU learnability diagnostic reaches 90.43% answer accuracy on its own MQAR protocol, while a saturated-state CPU benchmark reports a median recurrent decode rate of 143.71 tokens/s for a 363,385-parameter configuration. These measurements demonstrate executable mechanics and reproducible instrumentation only; broad quality or speed superiority requires the larger controlled evaluation program defined here.

Türkçe özet. ALUCLU, nedensel dizi modelleme için dört farklı ve açıkça sınırlı bellek yolunu aynı residual blokta birleştiren decoder-only bir araştırma mimarisidir: Gated Delta Rule-2 tabanlı yoğun recurrent fast-weight belleği, sabit pencereci exact softmax attention, sabit kapasiteli residual-surprise exact key-value cache ve yakın geçmişi exact factor, eski bölümleri düşük ranklı kapsül olarak tutan episodic bellek. Yollar aynı normalize edilmiş token temsilini okur ve SwiGLU katmanından önce token bazında öğrenilen katsayılarla birleştirilir. Makale, çalışan referans kodun denklemlerini, nedensellik sırasını, state ve hesap maliyetini, sıkıştırma hata defterini, bilinen başarısızlık kiplerini ve adil deney protokolünü ayrıntılı biçimde verir. Mevcut sonuçlar bir dünya rekoru veya SOTA iddiası değildir; uygulamanın çalıştığını, sınırlarının ölçülebildiğini ve daha büyük deneylere hazır bir doğruluk oracle'ı sunduğunu gösterir.

Keywords: bounded-state sequence modeling; recurrent language models; linear attention; associative memory; exact cache; episodic compression; long-context evaluation.

Contents

1	Introduction	5
1.1	Design question	5
1.2	Contributions	6
1.3	Evidence contract	6
1.4	Paper organization	6
2	Related Work	7
2.1	Linear attention and fast-weight memory	7
2.2	Selective state-space models	7
2.3	Recall limits and local/global hybrids	7
2.4	Residual-selected exact memory	8
2.5	Exact attention systems	8
2.6	Bounded low-rank compression	8
2.7	Sketch-based routing	8
2.8	Capacity and long-context evaluation	9
2.9	Positioning	9
3	Problem Formulation and Claim Contract	9
3.1	Causal streaming interface	9
3.2	Inclusive write-then-read semantics	9
3.3	Pareto objective	10
3.4	Information-capacity lower bound	10
3.5	Fair-comparison requirements	10
3.6	Progression gates	11
4	Architecture	11
4.1	Name and structural principle	11
4.2	Layer scheduling	11
4.3	Block data flow	11
4.4	Streaming causal stems	12
4.5	Token-wise branch fusion	12
4.6	State hierarchy	13
4.7	Sequence, chunk, and step paths	13
5	Memory Lanes and Theoretical Properties	13
5.1	Notation and block-level execution	13
5.2	Causal depthwise streaming stems	15
5.3	Gated Delta Rule-2 lane	15
5.4	Exact bounded local attention	16
5.5	Residual-surprise exact key-value cache	17
5.6	Bounded episodic associative memory	18
5.7	Thin-QR, small-core SVD compression	21
5.8	Compression error ledger	22
5.9	Token-wise branch fusion and SwiGLU	23
5.10	Global causality and chunk/step equivalence	24
5.11	Persistent-state and compute bounds	25
5.12	Finite-capacity boundary	26
5.13	Code correspondence and validation boundary	26

6	Reference Implementation	27
6.1	Package layout	27
6.2	Configuration validation	28
6.3	Numerical precision	28
6.4	State ownership and graph control	28
6.5	Determinism and discrete cache selection	28
6.6	Protocol provenance	29
6.7	Testing strategy	29
6.8	Reference-backend performance characteristics	29
6.9	Minimal recurrent use	30
7	Evaluation Methodology	30
7.1	Two non-interchangeable MQAR protocols	30
7.2	P0: correctness and small-scale decision gate	31
7.3	P1: model quality and long-context transfer	31
7.4	Baseline contract	32
7.5	Ablation program	32
7.6	Systems scenarios	33
7.7	Fused-kernel correctness gate	33
7.8	Memory traffic and energy	33
7.9	Result-artifact schema	33
8	Current Measured Evidence	34
8.1	Evidence boundary	34
8.2	Automated release gate	34
8.3	Small MQAR learnability diagnostic	34
8.4	Saturated-state CPU reference timing	34
8.5	What can be concluded	35
9	Limitations, Failure Modes, and Risk Controls	35
9.1	Memory is bounded because information is lost	35
9.2	Archive approximation	35
9.3	Abrupt ring eviction	35
9.4	Router error and conditioning	35
9.5	Surprise is not future utility	35
9.6	Hard selection limits learning	36
9.7	Training and numerical stability	36
9.8	Serving gaps	36
9.9	Evaluation gaps	36
9.10	Risk register	36
9.11	Responsible claim policy	37
10	Conclusion	37
A	Notation and Configuration Crosswalk	38
B	Persistent State Shapes	38
C	Operational Pseudocode	39
C.1	One block step	39
C.2	Exact-cache insertion	40
C.3	Episodic transition	40

D	Reproduction Commands	41
D.1	Environment and editable install	41
D.2	Complete portable release gate	41
D.3	Recorded diagnostic	41
D.4	Recorded saturated reference benchmark	41
E	Test-to-Claim Traceability	41
F	Suggested Citation and License	42
G	Release Checklist	42

1 Introduction

Long-context sequence modeling is constrained by three resources that cannot be optimized independently: representational capacity, memory traffic, and arithmetic work. Full causal self-attention preserves direct token-to-token access, but its key-value state grows with the processed sequence and its prefill attention matrix grows quadratically without additional structure. Recurrent and linear-attention models replace that growing state with a fixed-size summary, but any finite summary must eventually merge, overwrite, or discard independent information. Sparse caches preserve selected records exactly, yet a selection rule can retain the wrong records. Compressed episodic representations extend the age horizon, but compression creates a measurable approximation error.

ALUCLU starts from the premise that these are not four implementations of one memory abstraction. They are four different physical memory regimes:

Table 1: The four memory regimes used by ALUCLU. “Exact” is local to the stored representation and never means unbounded exact recall.

Regime	Primary role	Physical budget	Principal failure mode
Dense recurrence	Continuous global statistics	Fixed fast-weight matrix per layer	Interference and decay
Local softmax	Precise recent interactions	Last W projected keys and values	Hard recency horizon
Surprise cache	Sparse exceptional records	Highest-priority C_x projected KV records	Selection mismatch and cache freezing
Episodic capsules	Longer temporal organization	Current segment, K live segments, C rank- r archive capsules	Compression and overwrite

The architecture places these regimes in parallel inside selected transformer-like residual blocks. Every layer has a Gated Delta Rule-2 recurrent lane. Local, exact-cache, and episodic lanes are scheduled at configurable layer intervals. They all consume the same normalized token representation rather than forming a serial chain. Their outputs are independently normalized and combined with learned token-wise coefficients. This arrangement gives the model an explicit choice between a dense recurrent estimate, exact recent evidence, a sparse set of high-write-residual events, and temporally segmented compressed evidence.

1.1 Design question

The project asks a deliberately narrower and more testable question than “What is the universally best language-model architecture?”:

Can one build an executable causal reference architecture whose persistent decode state is bounded by configuration, whose heterogeneous memory losses are visible in state and metrics, and whose interfaces permit exact token-by-token auditing before specialized kernels or large-scale training?

This question matters because algorithmic asymptotics alone do not determine real performance. A theoretically constant-state recurrence can be slower than optimized attention if implemented as a Python token loop. A low-rank archive can have a long nominal horizon but poor task recall. A small cache can be “exact” for accepted keys while missing the one key a future query needs. Consequently, ALUCLU treats equations, code semantics, state bytes, benchmark protocols, and claim boundaries as parts of one engineering object.

1.2 Contributions

The present release makes the following concrete contributions.

1. It specifies and implements a four-lane decoder block with explicit scan, chunk, and recurrent-step interfaces and immutable external state containers.
2. It couples a Gated Delta Rule-2 write residual to a fixed-capacity exact KV cache. The cache retains the highest observed scalar residual priorities under a deterministic strict-replacement policy.
3. It implements a bounded episodic hierarchy with exact current/live factor sums, low-rank archive capsules, exact positive denominator statistics, a TensorSketch-assisted router, and a Frobenius error ledger.
4. It derives the persistent payload, retained temporal budget, and token-cost terms from the actual state fields instead of using an informal “constant memory” label.
5. It provides a mass-normalized routing construction that preserves the unweighted denominator mass in exact arithmetic when numerical safety clamps are inactive, while making the clamp qualification explicit.
6. It supplies an MIT-licensed Python package, command-line tools, protocol manifests, multi-lingual documentation, reproducible JSON evidence, and an automated release gate with 50 tests.
7. It defines a staged evaluation program that separates small correctness and learnability diagnostics from model-quality, accelerator, energy, and long-context claims.

1.3 Evidence contract

The strongest conclusions supported by the current artifact are mechanical: the code executes, its configured state is bounded, tested causal paths agree between full scans and recurrent stepping in evaluation mode, and the recorded small CPU experiments are reproducible under their declared protocols. The artifact does *not* establish frontier language-model quality, universal speed superiority, an optimal memory allocation, or a production GPU kernel.

We use three labels throughout the paper:

Implemented The behavior exists in the released source code and is connected to the public model path.

Measured A result file records the protocol, configuration, environment, and numerical outcome.

Proposed The experiment or optimization is fully specified but has not been executed as evidence for this release.

This vocabulary is intentionally conservative. It prevents a common category error in architecture research: presenting a promising mechanism, a Python prototype, or an asymptotic argument as though it were already an end-to-end systems result.

1.4 Paper organization

[Section 2](#) positions the design among recurrent, attention, hybrid, and long-context evaluation work. [Section 3](#) defines the causal interface and measurable Pareto contract. [Section 4](#) describes the block and execution order. [Section 5](#) gives the implemented equations, while [Section 5](#) derives state, complexity, and error properties. [Section 6](#) maps the specification to code. [Sections 7](#) and [8](#) separate the planned evaluation matrix from the measurements already available. [Section 9](#) lists failure modes and physical limits. The appendices provide configuration, state, test, and reproduction details.

2 Related Work

2.1 Linear attention and fast-weight memory

Kernelized linear attention rewrites causal attention as a recurrent accumulation of key-value statistics, replacing a context-length-dependent KV cache with a state whose size is fixed once feature and value dimensions are fixed [Katharopoulos et al., 2020]. This view exposes both the appeal and the limitation of the method: decode state stops growing, but many associations must share one finite matrix. Schlag et al. [2021] interpreted the matrix as fast weights and introduced a delta update that reads the value currently associated with a key and writes a residual rather than adding an unconditional outer product.

DeltaNet made this update practical at language-model scale through a chunkwise sequence-parallel algorithm and studied mixtures with sliding-window or sparse global attention [Yang et al., 2024]. Gated Delta Networks combined targeted correction with adaptive forgetting [Yang et al., 2025]. The Gated DeltaNet-2 technical report further separates channel-wise erase and write gates while retaining adaptive channel decay [Hatamizadeh et al., 2026]. Because the last work is a 2026 preprint, its architectural and empirical findings are attributed to its authors rather than treated as independently established.

ALUCLU uses an independently written token-recurrent update in this erase/write-separated design class. It is not a reproduction of the official chunkwise kernel and its CPU measurements do not inherit any reported accelerator performance.

2.2 Selective state-space models

Mamba demonstrated that input-dependent selective state-space parameters can provide content-sensitive sequence processing with a fixed recurrent state and a hardware-aware training form [Gu and Dao, 2024]. Structured State Space Duality connected semiseparable state-space operators to attention-like matrices and led to Mamba-2 [Dao and Gu, 2024]. Mamba-3 extends this line with richer discretization, complex-valued dynamics, and SISO/MIMO formulations; the current official record identifies it as an ICLR 2026 oral paper [Lahoti et al., 2026].

These models are essential bounded-state comparators, but ALUCLU makes a different decomposition. Its recurrent lane is a matrix-valued delta memory; exact recent context, selected exact records, and segmented compressed history remain explicit separate states. State-space dualities may inform kernelization and comparisons, but do not make the four lanes algebraically equivalent to Mamba-family layers.

2.3 Recall limits and local/global hybrids

Zoology introduced MQAR and showed that associative recall explains a substantial part of the gap observed between attention and efficient gated/convolutional models in its experiments [Arora et al., 2024a]. Based framed a recurrent-state-size/recall trade-off and combined linear attention with exact sliding-window attention, supported by IO-aware kernels [Arora et al., 2024b]. These results motivate ALUCLU’s exact local lane: recent interactions retain direct softmax access while the recurrent lane spends fixed state on compressed global structure.

Recent hybrids allocate memory differently. Kimi Linear combines a fine-grained delta recurrence with Multi-Head Latent Attention layers [Kimi Team, 2025]; because those attention layers retain a growing cache, the whole hybrid does not satisfy ALUCLU’s strict saturated-state contract. Native Hybrid Attention places recurrent long-term KV slots and recent sliding-window tokens in a shared softmax read [Du et al., 2026]. Infini-attention combines masked local attention with a bounded compressive memory [Munkhdalai et al., 2024]. ALUCLU instead keeps four reads semantically separate, normalizes them separately, and mixes their outputs. This avoids treating a positive-linear denominator as a softmax partition function and makes every lane independently ablatable.

2.4 Residual-selected exact memory

HOLA provides the closest direct precedent for an exact complement to delta memory: it retains a bounded set of KV pairs whose writes have large residual relative to the compressive state [Cui, 2026]. ALUCLU’s cache is motivated by the same high-level idea but defines its own policy. It uses the RMS of its Gated Delta Rule-2 write residual, detaches the scalar priority, applies deterministic strict top-capacity admission, and reads retained projected KV pairs with exact softmax. “Exact” therefore describes the attention over retained records, not the selection of arbitrary history. HOLA is a recent single-author preprint, so comparative conclusions require a matched replication.

Growing and sparse memories clarify the fixed-state boundary. Memory Caching stores recurrent checkpoints, so its usable memory grows with the sequence [Behrouz et al., 2026]. Sparse Delta Memory spends a fixed but much larger state on sparse explicit addressing [Cabannes et al., 2026]. Both are useful controls: one tests the benefit of relaxing bounded state; the other tests a different allocation of a fixed state budget.

2.5 Exact attention systems

FlashAttention computes exact softmax attention with IO-aware tiling, reducing high-bandwidth memory traffic without changing attention semantics [Dao et al., 2022]. FlashAttention-2 improves work partitioning and parallelism [Dao, 2024]. These works define the systems standard for implementing ALUCLU’s local and cache reads, but address a different axis from architectural state growth. An IO-efficient full-attention kernel still carries context-dependent KV state, while an asymptotically bounded recurrence can still be slow if implemented as a Python token loop.

2.6 Bounded low-rank compression

ALUCLU’s episodic lane keeps current and recent segments as exact factor sums for a positive linear-attention numerator, then merges older segments into a finite ring of rank-bounded capsules. Householder triangularization supplies the numerical basis for thin QR [Householder, 1958]. The SVD of the resulting small core represents the same factor product before truncation, and the Eckart–Young result characterizes the retained best-rank approximation for the local matrix [Eckart and Young, 1936]. Thin-SVD modification methods provide a streaming numerical precedent [Brand, 2006]; Frequent Directions is a principled deterministic alternative with different covariance guarantees [Ghashami et al., 2016].

This background supports a narrow claim: the discarded singular tail is an algebraic Frobenius error for the matrix being compressed. It is not a bound on task loss or arbitrary recall. ALUCLU keeps the positive denominator summary outside the truncated factors and records a conservative cumulative error ledger. When the finite ring wraps, overwritten history is unavailable.

2.7 Sketch-based routing

TensorSketch gives an explicit randomized feature map for polynomial kernels through compressed tensor-product features [Pham and Pagh, 2013]. PolySketchFormer demonstrates polynomial sketching in causal sequence models [Kacham et al., 2024]. ALUCLU uses the method more narrowly: it estimates polynomial similarity only among a finite set of current/live/archive slots, alongside a degree-one route.

A 2025 TensorSketch manuscript explicitly corrects the variance bound associated with the original account [Pham and Pagh, 2025]; consequently this paper does not repeat the earlier variance expression. Finite sketch width and replica count create routing error. Sketching neither reconstructs rank-truncated numerator information nor recovers an overwritten capsule.

2.8 Capacity and long-context evaluation

Task-specific separations show that some lookup problems require recurrent state growing linearly with input size even where relatively small Transformers suffice [Bhattamishra et al., 2024]. These results do not directly prove a task-accuracy bound for ALUCLU, but they reinforce the need for an honest finite-capacity contract.

Evaluation must also extend beyond one synthetic needle. RULER adds multi-needle retrieval, multi-hop tracing, and aggregation across controlled lengths [Hsieh et al., 2024]. NoLiMa removes easy literal overlap between questions and evidence [Modarressi et al., 2025]. BABI-Long distributes facts required for multi-step reasoning through long natural-text distractors [Kuratov et al., 2024]. HELMET shows why synthetic retrieval alone cannot stand in for diverse application-centric long-context performance [Yen et al., 2025]. ALUCLU evaluation should therefore report length and capacity curves for synthetic recall together with language-model loss and application-oriented task categories.

2.9 Positioning

Prior work supports three premises, not an ALUCLU performance conclusion. First, recurrent delta and state-space mechanisms provide bounded global state but face finite-capacity interference. Second, exact local or selected memory can complement a compressive state at an explicit cost in bytes and reads. Third, low-rank and sketching methods make compression measurable but cannot defeat information limits. ALUCLU’s contribution is the particular bounded four-lane allocation, its independently testable execution semantics, and its claim-and-measurement contract. Whether that composition improves a quality–latency–traffic–energy Pareto frontier remains an empirical question.

3 Problem Formulation and Claim Contract

3.1 Causal streaming interface

Let $x_{1:T}$ be a token sequence and let f_θ be a decoder-only model. At token t , the recurrent interface computes

$$(y_t, \mathcal{S}_t) = f_\theta(x_t, \mathcal{S}_{t-1}), \quad (1)$$

where y_t denotes logits or a hidden representation and $\mathcal{S}_t$ is the complete persistent state. Causality requires that y_t and $\mathcal{S}_t$ be functions only of $x_{1:t}$. For a fixed configuration $\mathcal{C}$, bounded persistent state means that there exists a finite quantity $M(\mathcal{C}, B)$ such that

$$\text{bytes}(\mathcal{S}_t) \leq M(\mathcal{C}, B) \quad \text{for every } t, \quad (2)$$

for batch size B . The bound may depend on width, layers, cache capacities, window size, ranks, data types, and batch size, but not on the number of already processed tokens.

Equation (2) concerns inference payload. It does not imply that training activation memory is independent of T : a naive autograd scan retains a graph whose size grows with the number of untruncated steps. Likewise, bounded logical tensor payload does not imply bounded allocator workspace or constant wall-clock latency at archive compression boundaries.

3.2 Inclusive write-then-read semantics

The implemented memory lanes use inclusive causal semantics. At time t , the current token is first admitted to, or used to update, the relevant state; the read then uses that updated state. Symbolically,

$$\mathcal{S}_t = \text{write}(x_t, \mathcal{S}_{t-1}), \quad y_t = \text{read}(x_t, \mathcal{S}_t). \quad (3)$$

For language modeling, loss is computed between logits $_{1:T-1}$ and labels $x_{2:T}$, so inclusive internal memory is compatible with next-token prediction. The ordering is important at local-window boundaries, exact-cache replacement events, and episodic segment seals; a read-before-write implementation would define a different model.

3.3 Pareto objective

No single scalar called “performance” captures the intended design target. For model and hardware configuration $\mathcal{C}$, define

$$\mathbf{J}(\mathcal{C}) = (Q_{\text{task}}, R_{\text{recall}}, \tau_{\text{prefill}}, \tau_{\text{decode}}, M_{\text{peak}}, M_{\text{state}}, E_{\text{token}}), \quad (4)$$

where Q_{task} is downstream quality, R_{recall} is controlled retrieval accuracy, τ_{prefill} and τ_{decode} are throughput, M_{peak} is peak runtime memory, M_{state} is persistent state payload, and E_{token} is energy per generated token. An architecture is interesting only if it shifts a relevant Pareto front under matched parameters, tokens, optimizer, precision, compiler, hardware, and quality constraints.

For this reason, the current release does not optimize Equation (4) into one headline score. It exposes the budgets and protocols needed to measure each coordinate independently.

3.4 Information-capacity lower bound

Suppose a finite state contains at most Q bits and a task asks the model to recover N independent values drawn uniformly from an alphabet $\mathcal{V}$, with query keys known externally. The values alone have $N \log_2 |\mathcal{V}|$ bits of entropy. Lossless recovery for all possible assignments therefore requires

$$Q \geq N \log_2 |\mathcal{V}|, \quad \text{or equivalently} \quad N \leq \frac{Q}{\log_2 |\mathcal{V}|}. \quad (5)$$

The bound is optimistic: it ignores floating-point noise, query ambiguity, router collisions, rank constraints, and the computational limits of the readout. It nevertheless rules out unbounded exact arbitrary recall from a fixed-precision state. ALUCLU therefore treats forgetting and approximation as explicit budgeted operations rather than hiding them behind an unlimited context-length label.

3.5 Fair-comparison requirements

An end-to-end comparison is valid only when the following conditions are reported and, where relevant, matched:

- training data, tokenizer, token count, data order, and contamination controls;
- parameter count, effective depth and width, optimizer, schedule, batch tokens, and gradient clipping;
- inference precision, quantization, batch size, prompt length, generation length, and sampling policy;
- software versions, compiler flags, kernel implementation, device, power mode, and thermal state;
- whether model loading, random-token generation, compilation, synchronization, and state allocation occur inside the timed region;
- accuracy or loss at the throughput operating point;
- median and dispersion across repeated runs, not only a best sample.

The portable reference backend is primarily a semantic oracle. It should be compared with optimized systems only after the same recurrence has a specialized implementation, and both semantic equality and numerical drift have been tested against the oracle.

3.6 Progression gates

The project uses four evidence gates.

1. **Mechanics gate:** unit invariants, causality, scan/step equivalence, state saturation, serialization policy, and finite gradients.
2. **Learnability gate:** multiple seeds and controlled synthetic tasks, including MQAR protocols whose target alignment is frozen.
3. **Quality gate:** matched-token pretraining and long-context downstream evaluation against serious recurrent, attention, and hybrid baselines.
4. **Systems gate:** fused kernels, profiler evidence, peak memory, decode and prefill throughput, energy, and quality-matched comparisons on target accelerators.

The present v0.1.0 artifact passes the mechanics gate for its tested domain and contains one small learnability diagnostic. The larger quality and systems gates remain proposed work.

4 Architecture

4.1 Name and structural principle

ALUCLU expands to **Adaptive Layered Unified Context with Learned Updates**. “Layered” denotes scheduled placement across depth and the separation of physical memory regimes. “Unified context” denotes a single token representation assembled from all active lanes. “Learned updates” refers both to recurrent write dynamics and token-wise fusion. The name is written with ASCII **C** in code, package names, and URLs.

The unit of composition is `AlucluBlock`. Given input $x_t^{(\ell)}$, the block first computes

$$h_t^{(\ell)} = \text{RMSNorm}_{\text{mix}}\left(x_t^{(\ell)}\right). \quad (6)$$

Every active memory lane reads $h_t^{(\ell)}$ in parallel. A lane never consumes another lane’s output as its input. This keeps lane semantics separable and lets each state transition be audited against the same token.

4.2 Layer scheduling

Every one of the L layers contains a Gated Delta lane. The other lanes are periodic:

$$N_{\text{local}} = \left\lfloor \frac{L}{e_{\text{local}}} \right\rfloor, \quad (7)$$

$$N_{\text{epi}} = \left\lfloor \frac{L}{e_{\text{epi}}} \right\rfloor, \quad (8)$$

$$N_{\text{exact}} = \left\lfloor \frac{L}{e_{\text{exact}}} \right\rfloor, \quad (9)$$

when the corresponding configuration exists. Here each e is the “every” interval. This schedule permits a recurrent backbone at every depth while concentrating higher-state or higher-latency lanes at selected blocks.

4.3 Block data flow

[Figure 1](#) shows the implemented data flow. The residual-surprise cache receives its scalar insertion priority from the Gated Delta write residual, but its query, key, and value projections still read $h_t^{(\ell)}$.

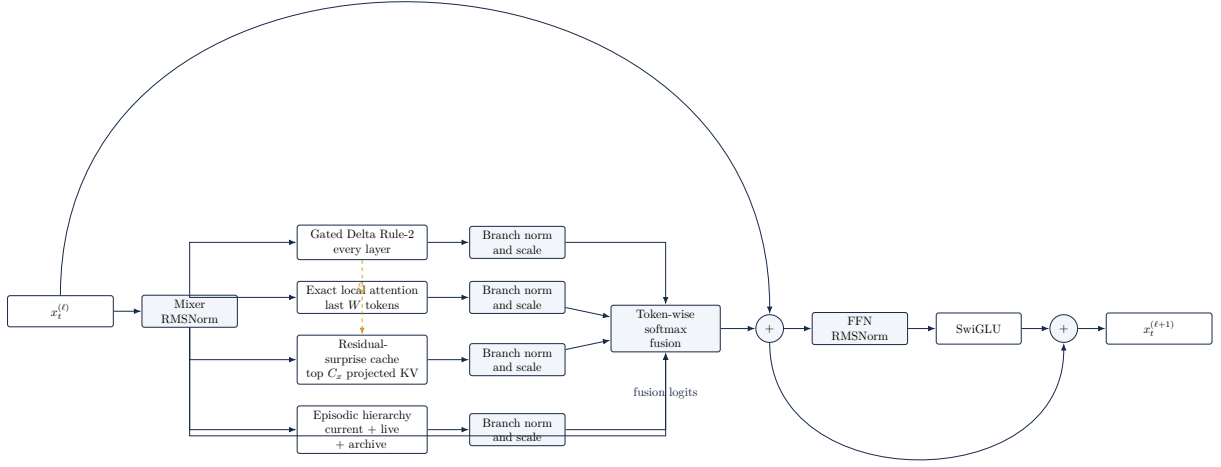


Figure 1: ALUCLU block. Solid arrows carry vectors. The dashed arrow carries the scalar Gated Delta write-surprise signal used by the exact-cache replacement policy. Every memory lane otherwise reads the same normalized token.

4.4 Streaming causal stems

The Gated Delta and episodic lanes each begin with an independent causal depthwise convolution of kernel width κ . For an input chunk,

$$c = [h_{\text{history}}; x_{1:T}], \quad (10)$$

$$\tilde{h}_{1:T} = \text{DWConv1D}(c), \quad (11)$$

$$h'_{\text{history}} = c_{-(\kappa-1):}. \quad (12)$$

The persistent history has $B(\kappa - 1)d$ elements. Kernels are initialized to identity by setting all weights to zero except the final causal tap, which is one, with zero bias. Thus adding the stem does not perturb the initial token path while allowing training to learn short causal filtering.

4.5 Token-wise branch fusion

Let $b_{t,j}$ be active branch j 's output. Each branch has a separate normalizer and learned scalar:

$$\tilde{b}_{t,j} = s_j \text{RMSNorm}_j(b_{t,j}). \quad (13)$$

For more than one branch, the mixture is

$$\alpha_t = \text{softmax}(W_f h_t + b_f), \quad (14)$$

$$f_t = \sum_j \alpha_{t,j} \tilde{b}_{t,j}, \quad \alpha_{t,j} > 0, \quad \sum_j \alpha_{t,j} = 1. \quad (15)$$

The coefficients form a convex combination of the *scaled normalized* branches. Because s_j is unconstrained, f_t is not necessarily in the convex hull of the raw $b_{t,j}$. The projection is initialized to zero, so the initial coefficients are uniform; all branch scales begin at one.

The residual and feed-forward path is

$$x'_t = x_t + \text{Dropout}(f_t), \quad (16)$$

$$(g_t, v_t) = \text{split}(W_{\text{in}} \text{RMSNorm}(x'_t)), \quad (17)$$

$$u_t = W_{\text{out}}(\text{SiLU}(g_t) \odot v_t), \quad (18)$$

$$x_t^{\text{out}} = x'_t + \text{Dropout}(u_t). \quad (19)$$

4.6 State hierarchy

The public model state is a tuple of per-block states. A block state contains the always-present Gated Delta state and optional local, episodic, and exact-cache states. The state is returned explicitly by `forward` and `step`; it is never hidden inside the module or reset implicitly.

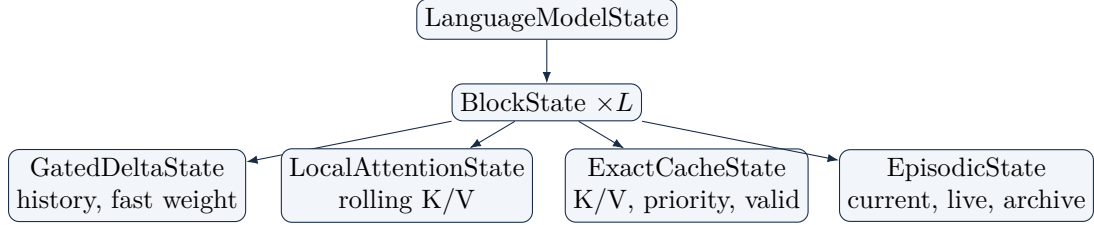


Figure 2: Logical persistent-state ownership. Optional branches appear only in scheduled layers.

4.7 Sequence, chunk, and step paths

The same token recurrence backs three interfaces:

Full sequence. Consume a complete $B \times T$ tensor from an optional incoming state and return outputs plus final state.

Chunk. Invoke the full-sequence path repeatedly while carrying the returned state. This is useful for truncated backpropagation or streaming prefill.

Step. Consume one token per batch item and return one-token logits and state.

In evaluation mode, the tests require these partitions to produce equivalent outputs and terminal state within the declared numerical tolerance. Training mode with dropout is not partition invariant because different partitions consume random numbers in a different order.

5 Memory Lanes and Theoretical Properties

This section specifies the mathematical object implemented by the current ALUCLU reference backend. The specification is intentionally narrower than an abstract family of possible models: every state variable, update order, and stabilizer below has a direct counterpart in `src/aluclu`. The implementation is token recurrent and uses explicit caller-owned state. It is therefore useful to distinguish three kinds of statement throughout the section:

1. algebraic identities of the real-arithmetic recurrence;
2. conditional identities whose hypotheses state when numerical clamps are inactive; and
3. empirical invariants exercised by the finite-precision test suite.

In particular, “exact” denotes the absence of rank truncation for a retained projected key–value record or live factor. It does not denote unbounded, lossless recall.

5.1 Notation and block-level execution

Let B be the batch size, T the number of tokens processed by one call, d the model width, and L the number of residual blocks. Table 2 maps the remaining symbols to the configuration fields used by the implementation.

Table 2: Symbols and their implementation counterparts.

Symbol	Code field	Meaning
H, d_k, d_v	<code>n_heads,</code> <code>head_key_dim</code> <code>head_value_dim</code>	Number of Gated Delta heads and per-head key/value widths.

Symbol	Code field	Meaning
W	<code>local_window</code>	Capacity of each local-attention rolling key-value window.
H_x, d_x, C_x	<code>ExactCacheConfig.n_heads,</code> <code>d/H_x, capacity</code>	Exact-cache head count, head width, and number of retained records.
S, K, C, A, r	<code>segment_size,</code> <code>live_segments,</code> <code>archive_slots,</code> <code>archive_segments_per_slot,</code> <code>archive_rank</code>	Episodic segment length, exact-live count, archive-capsule count, segments per capsule, and archive rank.
d_r, p, m, J	<code>router_dimension,</code> <code>router_degree,</code> <code>sketch_dim,</code> <code>n_sketches</code>	Route width, TensorSketch degree, sketch width, and number of sketches.
κ_g, κ_e	<code>GatedDeltaConfig.conv_kernel,</code> <code>EpisodicMemoryConfig.conv_kernel</code>	Causal depthwise-stem kernel sizes.
$f_{\min}, f_{\max}, \varepsilon$	<code>feature_floor,</code> <code>feature_ceiling,</code> <code>eps</code>	Episodic feature bounds and numerical lower clamp.
b_x	<code>x.element_size()</code>	Bytes per model-state scalar.

For block ℓ , let $x_{\ell,t} \in \mathbb{R}^d$ denote its input at token t , and define

$$h_{\ell,t} = \text{RMSNorm}(x_{\ell,t}). \quad (20)$$

Every block contains a Gated Delta lane. The local, episodic, and residual-surprise lanes are present when, respectively,

$$\ell \equiv 0 \pmod{\text{local_attention_every}}, \quad (21)$$

$$\ell \equiv 0 \pmod{\text{episodic_every}}, \quad (22)$$

$$\ell \equiv 0 \pmod{\text{exact_cache_every}}, \quad (23)$$

using one-based layer indices, with the additional requirement that the episodic or exact-cache configuration exists. Consequently,

$$N_{\text{local}} = \left\lfloor \frac{L}{\text{local_attention_every}} \right\rfloor, \quad (24)$$

$$N_{\text{epi}} = \mathbf{1}_{\{\text{episodic configured}\}} \left\lfloor \frac{L}{\text{episodic_every}} \right\rfloor, \quad (25)$$

$$N_{\text{exact}} = \mathbf{1}_{\{\text{exact cache configured}\}} \left\lfloor \frac{L}{\text{exact_cache_every}} \right\rfloor. \quad (26)$$

All active lanes read the same $h_{\ell,t}$; their outputs are not chained. The Gated Delta lane additionally emits one scalar write-surprise value per batch item and token. The exact-cache lane consumes this scalar for its hard insertion decision, but it computes its own query, key, and value projections. Each memory lane performs its state update before reading at the same token. Thus, the implemented convention is inclusive causal, $j \leq t$.

5.2 Causal depthwise streaming stems

The Gated Delta and episodic lanes each own an independent `StreamingDepthwiseConv1d`. For kernel width κ , input chunk $x_{1:T}$, and incoming history $c \in \mathbb{R}^{B \times (\kappa-1) \times d}$, the operation is

$$\bar{x} = [c; x_{1:T}], \quad (27)$$

$$\tilde{x}_{1:T} = \text{DepthwiseConv1d}_\kappa(\bar{x}), \quad (28)$$

$$c' = \bar{x}_{-(\kappa-1):}. \quad (29)$$

The history therefore stores exactly $B(\kappa-1)d$ model-dtype elements. The convolution is initialized as an identity map: all weights are zero except the most recent causal tap, which is one, and the bias is zero.

Proposition 1 (Chunk compositionality of the causal stem) *In exact arithmetic, processing a sequence by (29) in any ordered partition produces the same output sequence and terminal history as processing it in one call.*

Proof sketch. At every boundary the state c' is exactly the suffix of length $\kappa-1$ required by the next causal convolution. Each output at time t is therefore evaluated on the same ordered κ -token context under every partition. Induction over chunk boundaries proves equality. $\square$

5.3 Gated Delta Rule-2 lane

Suppress the block, batch, and head indices temporarily. The lane applies its own streaming stem to h_t , producing $\tilde{h}_t$, and then forms

$$q_t = \frac{W_q \tilde{h}_t}{\max(\|W_q \tilde{h}_t\|_2, \varepsilon)} \in \mathbb{R}^{d_k}, \quad (30)$$

$$k_t = \frac{W_k \tilde{h}_t}{\max(\|W_k \tilde{h}_t\|_2, \varepsilon)} \in \mathbb{R}^{d_k}, \quad (31)$$

$$v_t = W_v \tilde{h}_t \in \mathbb{R}^{d_v}, \quad (32)$$

$$e_t = \text{sigmoid}(W_e \tilde{h}_t + b_e) \in (0, 1)^{d_k}, \quad (33)$$

$$w_t = \text{sigmoid}(W_w \tilde{h}_t + b_w) \in (0, 1)^{d_v}. \quad (34)$$

Here e_t is the erase gate and w_t is the write gate. Let $a = \exp(\text{decay_log_scale}) > 0$. The token-dependent decay is

$$\log \gamma_t = -a \odot \text{softplus}(W_\gamma \tilde{h}_t + \text{decay_offset}), \quad (35)$$

$$\gamma_t = \max\{\exp(\log \gamma_t), \text{decay_min}\}, \quad (36)$$

where the maximum is elementwise. In real arithmetic, $\text{decay_min} \leq \gamma_{t,j} < 1$.

For each head, the persistent fast-weight matrix is $F_{t-1} \in \mathbb{R}^{d_k \times d_v}$. The implemented recurrence is

$$\bar{F}_t = \text{diag}(\gamma_t) F_{t-1}, \quad (37)$$

$$a_t = e_t \odot k_t, \quad (38)$$

$$\hat{v}_t = \bar{F}_t^\top a_t, \quad (39)$$

$$v_t^{\text{target}} = w_t \odot v_t, \quad (40)$$

$$\Delta v_t = v_t^{\text{target}} - \hat{v}_t, \quad (41)$$

$$F_t = \bar{F}_t + k_t (\Delta v_t)^\top, \quad (42)$$

$$o_t = F_t^\top q_t. \quad (43)$$

The outer-product update and read use the newly written F_t , rather than F_{t-1} . With head indices restored, the write-surprise scalar is

$$\xi_t = \frac{\left(\sum_{h=1}^H \sum_{j=1}^{d_v} (\Delta v_{t,h,j})^2\right)^{1/2}}{\sqrt{H d_v}} = \sqrt{\text{mean}_{h,j}[(\Delta v_{t,h,j})^2]}. \quad (44)$$

Thus ξ_t is an RMS write residual, not an unnormalized Euclidean norm. The per-head output is gated and projected:

$$g_t = W_{\text{out}} \text{concat}_{h=1}^H \left[o_{t,h} \odot \text{SiLU}(W_o \tilde{h}_t)_h \right]. \quad (45)$$

The implementation evaluates the fast-weight state, decay path, and recurrence in fp32 even under mixed precision, then casts the stacked recurrent output back to the model dtype before the output projection.

Proposition 2 (Fixed recurrent state and inclusive causality) *For fixed H, d_k, d_v, κ_g, d , the Gated Delta lane carries*

$$(\kappa_g - 1)d \text{ model-dtype elements and } H d_k d_v \text{ fp32 elements} \quad (46)$$

per batch item, independently of the number of previously processed tokens. Its output at t is a function only of inputs through t , and it includes the update induced by token t .

Proof sketch. The only persistent tensors are the bounded convolution history and one $d_k \times d_v$ fast-weight matrix per head. Every quantity on the right-hand side of (43) depends on F_{t-1} and the current stem output. The read uses F_t , establishing write-then-read causality. $\square$

5.4 Exact bounded local attention

Let H_ℓ be the number of local-attention heads and $d_h = d/H_\ell$. At token t , projected k_t, v_t are appended before the read and the cache is truncated to its last W entries:

$$K_t = [K_{t-1}; k_t]_{-W:}, \quad V_t = [V_{t-1}; v_t]_{-W:}. \quad (47)$$

For a retained position j and head h ,

$$s_{t,j,h} = \frac{q_{t,h}^\top k_{j,h}}{\sqrt{d_h}} - \text{softplus}(\text{slope_raw}_h)(t - j), \quad (48)$$

where the implementation constructs $t - j$ as the reverse distance within the rolling window. It then computes

$$\alpha_{t,:,h} = \text{softmax}(s_{t,:,h}), \quad (49)$$

$$\ell_{t,h} = \sum_{j \in \mathcal{W}_t} \alpha_{t,j,h} v_{j,h}, \quad \mathcal{W}_t = \{\max(1, t - W + 1), \dots, t\}. \quad (50)$$

The softmax is explicitly evaluated in fp32 and cast back to the model dtype. Attention-weight dropout is applied only in training mode.

Proposition 3 (Local exactness and bound) *The local lane computes ordinary causal softmax attention, with the learned linear recency bias in (48), over exactly the retained window $\mathcal{W}_t$. At saturation its persistent key-value payload is*

$$F_{\text{local}} = 2dW \quad (51)$$

model-dtype elements per batch item, and its attention read costs $\mathcal{O}(dW)$ per token, excluding dense projections.

Proof sketch. Equation (47) retains no index earlier than $t - W + 1$, and it inserts the current key and value before (50). The softmax is not kernelized or low-rank approximated. Two tensors of shape $H_\ell \times W \times d_h$ yield $2dW$ elements. $\square$

5.5 Residual-surprise exact key-value cache

The exact cache has independent projections

$$q_t^x = W_q^x h_t, \quad k_t^x = W_k^x h_t, \quad v_t^x = W_v^x h_t, \quad (52)$$

split into H_x heads of width $d_x = d/H_x$. It receives the scalar ξ_t from (44); no separate learned priority projection is present.

5.5.1 Hard insertion rule

For each batch item, the state contains C_x key slots, value slots, priorities, and valid bits. Let

$$\mathcal{E}_t = \{j : \neg \text{valid}_{t-1,j}\}. \quad (53)$$

If $\mathcal{E}_t \neq \emptyset$, the implementation selects the first invalid slot. Otherwise it selects

$$j_{\min} = \arg \min_{1 \leq j \leq C_x} \text{priority}_{t-1,j}. \quad (54)$$

The comparison priority is detached from autograd and converted to the priority dtype:

$$\hat{\xi}_t = \text{stopgrad}(\xi_t). \quad (55)$$

The write predicate is

$$\text{write}_t = \mathbf{1}\{\mathcal{E}_t \neq \emptyset\} \vee \mathbf{1}\{\hat{\xi}_t > \text{priority}_{t-1,j_{\min}}\}. \quad (56)$$

The inequality is strict: an equal-priority arrival does not replace the older retained record. All heads write the current projected key and value to the same selected slot.

Proposition 4 (Stable top- C_x priority invariant) *Assume finite priorities and process one batch item in temporal order. After t updates, the cache contains $\min(C_x, t)$ records whose priorities are the largest priorities seen so far, with earlier records retained at the cutoff when priorities tie.*

Proof sketch. Before saturation every arrival occupies one empty position. After saturation, an arrival below or equal to the current minimum cannot belong to a strictly better top- C_x set and is rejected. An arrival above the minimum replaces exactly one minimum. Induction on t gives the invariant; strict comparison yields stable earlier-tie retention. $\square$

5.5.2 Write-then-read exact softmax

Insertion precedes the current read. For valid slot j ,

$$s_{t,h,j}^x = \frac{(q_{t,h}^x)^\top k_{j,h}^x}{\sqrt{d_x}}, \quad (57)$$

$$\alpha_{t,h,:}^x = \text{softmax}(s_{t,h,:}^x), \quad (58)$$

$$o_{t,h}^x = \sum_{j:\text{valid}_j} \alpha_{t,h,j}^x v_{j,h}^x. \quad (59)$$

Invalid positions are masked with the minimum finite value of the score dtype. The first token is always inserted because capacity is positive and an empty slot exists, so every read has at least one valid entry. Half and bfloat16 scores use fp32 softmax; float32 and float64 scores retain their score dtype. The current record participates in its read if accepted. If rejected, its query still reads the retained past, but its key and value are absent.

The selection path is deliberately non-differentiable: $\hat{\xi}_t$ is detached, and integer indexing plus one-hot selection implements the replacement. Accepted key/value projections receive gradient through (59); the cache loss does not backpropagate through the discrete ranking decision into ξ_t .

Remark 1 (Meaning of exactness) *The cache stores the projected k_t^x, v_t^x of accepted records without rank truncation or averaging. It does not store all tokens, raw hidden states, token identifiers, or any record rejected or later evicted by (56).*

Its logical state size per batch item is

$$F_{\text{exact}} = 2C_x d + 2C_x. \quad (60)$$

For ordinary fp16, bfloat16, or fp32 model state, priority is fp32 and valid is Boolean, so the byte payload is

$$\text{bytes}_{\text{exact}, B=1} = 2b_x C_x d + 5C_x. \quad (61)$$

In the float64 validation path priority is also float64, replacing $5C_x$ by $9C_x$. Full-shape `torch.where` updates and the softmax read both cost $\mathcal{O}(C_x d)$ per token, excluding dense projections.

5.6 Bounded episodic associative memory

The episodic lane applies its own causal stem to h_t , yielding $\tilde{h}_t^e$, and uses a bounded positive feature map

$$\phi(a) = f_{\min} + (f_{\max} - f_{\min}) \text{sigmoid}(a), \quad 0 < f_{\min} < f_{\max}. \quad (62)$$

The implementation computes

$$q_t^+ = \phi(W_q^e \tilde{h}_t^e), \quad (63)$$

$$k_t^+ = \phi(W_k^e \tilde{h}_t^e), \quad (64)$$

$$v_t^e = W_v^e \tilde{h}_t^e. \quad (65)$$

Componentwise,

$$f_{\min} \leq \phi(a)_j \leq f_{\max}, \quad (66)$$

where equality can occur through floating-point sigmoid saturation.

5.6.1 Separate numerator and denominator summaries

For a retained token set $\mathcal{I}_i$, slot i represents

$$M_i = \sum_{t \in \mathcal{I}_i} v_t^e (k_t^+)^{\top}, \quad (67)$$

$$z_i = \sum_{t \in \mathcal{I}_i} k_t^+. \quad (68)$$

The numerator is stored in factors

$$M_i = L_i^{\top} R_i, \quad L_i, R_i \in \mathbb{R}^{f_i \times d}, \quad (69)$$

corresponding to the code fields `left` and `right`. For an exact current or live segment, one factor row corresponds to one token. The slot read is

$$n_i(q_t^+) = M_i q_t^+ = L_i^{\top} (R_i q_t^+), \quad (70)$$

$$d_i(q_t^+) = z_i^{\top} q_t^+. \quad (71)$$

The denominator is never reconstructed from R_i . After SVD truncation, R_i is a signed low-rank factor and is not a collection of positive keys. The independent z_i is therefore the sole valid denominator summary.

Proposition 5 (Positive denominator and compression separation) *Every nonempty retained slot satisfies*

$$d_i(q_t^+) > 0 \quad (72)$$

in real arithmetic. Rank truncation of M_i changes only the numerator in (71); it does not change z_i or d_i .

Proof sketch. Every component of every retained k_t^+ and of q_t^+ is positive by (62). Hence z_i is componentwise positive and its inner product with q_t^+ is positive. The compression routines replace only `left` and `right`; they copy or add z_i directly. $\square$

5.6.2 Route features and TensorSketch

The route vector is

$$u_t = \frac{W_r \tilde{h}_t^e}{\max(\|W_r \tilde{h}_t^e\|_2, \varepsilon)} \in \mathbb{R}^{d_r}. \quad (73)$$

Thus $\|u_t\|_2 = 1$ when the pre-normalization norm is at least ε , and $\|u_t\|_2 \leq 1$ otherwise.

For sketch $j \in \{1, \dots, J\}$, degree factor $a \in \{1, \dots, p\}$, bucket $b \in \{0, \dots, m-1\}$, fixed hash $h_{j,a}$, and fixed sign map $s_{j,a}$, define

$$\text{CS}_{j,a}(u)[b] = \sum_{\ell: h_{j,a}(\ell)=b} s_{j,a}(\ell) u_\ell. \quad (74)$$

The implemented TensorSketch feature is

$$\psi_j(u) = \text{IFFT} \left(\prod_{a=1}^p \text{FFT}(\text{CS}_{j,a}(u)) \right) \in \mathbb{R}^m. \quad (75)$$

Hashes and signs are generated once from `router_seed` and registered as persistent buffers. A slot stores additive summaries

$$r_i = \sum_{t \in \mathcal{I}_i} u_t, \quad (76)$$

$$\Psi_{i,j} = \sum_{t \in \mathcal{I}_i} \psi_j(u_t), \quad (77)$$

$$c_i = |\mathcal{I}_i|. \quad (78)$$

The sketch is used only to score slots. Finite m and J permit collisions, and the implementation does not claim a collision-free identity store. Configuration validation requires odd J , making the median below unambiguous.

5.6.3 Mass-weighted routing and its clamp condition

For route query u_q and slot i , the linear and polynomial scores are

$$\ell_i = \frac{u_q^\top r_i}{\sqrt{\max(c_i, 1)}}, \quad (79)$$

$$\rho_i = \frac{\text{median}_{j=1, \dots, J} [\psi_j(u_q)^\top \Psi_{i,j}]}{\sqrt{\max(c_i, 1)}}. \quad (80)$$

Let

$$(\pi_1, \pi_p) = \text{softmax}(\text{mix_logits}), \quad s_i = \pi_1 \ell_i + \pi_p \rho_i. \quad (81)$$

The signed, bounded temperature is

$$\theta = \text{max_temperature} \tanh(\text{temperature_raw}). \quad (82)$$

The implementation does not constrain θ to be positive.

It is important to state the numerical stabilizers exactly. With

$$D_0 = \sum_i d_i, \quad (83)$$

$$D_\varepsilon = \max(D_0, \varepsilon), \quad (84)$$

$$\bar{s} = \frac{\sum_i s_i d_i}{D_\varepsilon}, \quad (85)$$

$$\eta_i = \text{clip}(\theta(s_i - \bar{s}), -L_w, L_w), \quad (86)$$

$$\tilde{w}_i = \exp(\eta_i) > 0, \quad (87)$$

$$c_\varepsilon = \max\left(\frac{\sum_i \tilde{w}_i d_i}{D_\varepsilon}, \varepsilon\right), \quad (88)$$

$$w_i = \frac{\tilde{w}_i}{c_\varepsilon}, \quad L_w = \text{max_log_weight}, \quad (89)$$

the routed numerator and returned denominator are

$$N_{\text{route}} = \sum_i w_i n_i, \quad (90)$$

$$D_{\text{returned}} = D_0, \quad (91)$$

$$y_t^e = \frac{N_{\text{route}}}{\max(D_{\text{returned}}, \varepsilon)}. \quad (92)$$

The code intentionally does not multiply the returned denominator by w_i .

Proposition 6 (Conditional denominator-mass conservation) *If both lower clamps in (89) are inactive, namely*

$$D_0 \geq \varepsilon \quad \text{and} \quad \frac{\sum_i \tilde{w}_i d_i}{D_0} \geq \varepsilon, \quad (93)$$

then

$$w_i > 0, \quad \sum_i w_i d_i = D_0. \quad (94)$$

If additionally $\theta = 0$, then $w_i = 1$ for every slot.

Proof sketch. Under (93),

$$c_\varepsilon = \frac{\sum_i \tilde{w}_i d_i}{D_0}.$$

Substitution into $w_i = \tilde{w}_i / c_\varepsilon$ yields $\sum_i w_i d_i = D_0$; positivity follows from the exponential and positive normalizer. When $\theta = 0$, every $\eta_i = 0$, hence $\tilde{w}_i = 1$ and $c_\varepsilon = 1$. $\square$

Remark 2 (Active-clamp behavior) *When either condition in (93) fails, the implemented stabilized formula remains (89), but (94) is not an unconditional identity. For example, if only the first clamp is active and the second is inactive, then $\sum_i w_i d_i = D_\varepsilon$, not necessarily D_0 . This distinction is why all exact mass-conservation claims in this paper carry the inactive-clamp hypothesis. The unit test exercises positive masses well above ε and verifies the equality to floating-point tolerance.*

5.6.4 Current, live, and archive lifecycle

Each token first creates a one-row slot and is appended to **current**. When the current factor count reaches S , the completed slot moves to the end of the exact **live** tuple and **current** becomes empty. If this produces more than K live segments, the oldest live segment is evicted. When $C = 0$, that segment is forgotten. Otherwise it enters a temporal archive ring:

1. the first segment of a capsule is compressed to rank r ;
2. further incoming segments are merged into the active capsule until it contains A segments;
3. the cursor then advances to the next capsule; and
4. once all C positions exist, advancing overwrites the next capsule and erases its former contents.

The physical tuple order is not required to be chronological; the cursor identifies the active ring position.

The method `maximum_retained_tokens()` returns the conservative allocation-style horizon

$$H_{\max} = S(1 + K + CA) = \underbrace{S}_{\text{current budget}} + \underbrace{KS}_{\text{exact live}} + \underbrace{CAS}_{\text{compressed archive}}. \quad (95)$$

Because a current segment is sealed immediately at S ,

$$0 \leq |\mathbf{current}| < S, \quad N_{\text{retained}} \leq H_{\max} - 1 \quad (96)$$

for an actually reachable state. After a full-ring overwrite, the number of archived segments falls from CA to $(C - 1)A + 1$, then rises as the new active capsule fills. Hence the age horizon has a sawtooth variation of up to $A - 1$ archived segments.

Proposition 7 (Bounded episodic lifecycle) *For every stream length,*

$$|\mathbf{live}| \leq K, \quad |\mathbf{archive}| \leq C, \quad \mathbf{archive_cursor_segments} \leq A, \quad (97)$$

and (96) holds. Current and live numerator factors are algebraically exact sums for their retained tokens; archive numerators have rank at most r ; overwritten capsules have no remaining representation.

Proof sketch. Sealing resets current before it can exceed $S - 1$. Live overflow immediately evicts its oldest element, maintaining K . Archive insertion either merges into the current capsule, appends while fewer than C capsules exist, or overwrites one existing capsule. The cursor advances after A segments. These transitions establish (97) by induction; counting their maximum represented tokens yields (96). $\square$

5.7 Thin-QR, small-core SVD compression

The archive compresses a factor pair without materializing a dense $d \times d$ numerator. Suppress the batch index and let

$$L = \mathbf{left}^\top \in \mathbb{R}^{d \times F}, \quad R = \mathbf{right}^\top \in \mathbb{R}^{d \times F}, \quad M = LR^\top. \quad (98)$$

Reduced QR decompositions give

$$L = Q_L R_L, \quad R = Q_R R_R. \quad (99)$$

The code performs an SVD only on

$$C_{\text{core}} = R_L R_R^\top = U_c \Sigma V_c^\top. \quad (100)$$

If $p_c = \min(d, F)$, then $C_{\text{core}} \in \mathbb{R}^{p_c \times p_c}$. Let

$$r_{\text{eff}} = \min(r, \text{len}(\Sigma)). \quad (101)$$

The returned factors are

$$\widehat{L}_{\text{rows}} = \left(Q_L U_{c,:r_{\text{eff}}} \Sigma_{:,r_{\text{eff}}}^{1/2} \right)^\top, \quad (102)$$

$$\widehat{R}_{\text{rows}} = \left(Q_R V_{c,:r_{\text{eff}}} \Sigma_{:,r_{\text{eff}}}^{1/2} \right)^\top. \quad (103)$$

Zero rows pad each factor to exactly r , keeping archive shapes static. For half or bfloat16 input the QR and SVD work in fp32 before results are cast back; float32 and float64 retain their input work dtype.

Proposition 8 (Best rank- r numerator in exact arithmetic) *Ignoring finite-precision QR/SVD and the final dtype cast, the reconstructed*

$$\widehat{M} = \widehat{L}_{\text{rows}}^\top \widehat{R}_{\text{rows}} \quad (104)$$

is a best rank- r_{eff} approximation to M in Frobenius norm, and

$$\|M - \widehat{M}\|_F = \delta = \left(\sum_{j > r_{\text{eff}}} \sigma_j(C_{\text{core}})^2 \right)^{1/2}. \quad (105)$$

If $r \geq \text{rank}(M)$, the compression is algebraically exact.

Proof sketch. Reduced QR factors have orthonormal columns, so left and right multiplication by $Q_L, Q_R^\top$ preserves all nonzero singular values and Frobenius norms. Truncating the SVD of C_{core} therefore gives the Eckart–Young best-rank approximation to M . Splitting $\Sigma^{1/2}$ symmetrically reconstructs that truncation. $\square$

In a typical merge, an existing rank- r capsule is concatenated with an exact S -row live segment, so $F \leq r + S$ and

$$p_c = \min(d, F) \leq \min(d, r + S). \quad (106)$$

One merge costs approximately

$$\mathcal{O}(dFp_c + p_c^3). \quad (107)$$

Since a live overflow occurs at most once every S tokens, its steady-state amortized cost is

$$\mathcal{O}\left(\frac{d(r + S)p_c + p_c^3}{S}\right) \text{ per token.} \quad (108)$$

The core is genuinely small only when $r + S \ll d$. Configuration validation enforces $r \leq d$, but does not enforce $r + S < d$; the core can therefore grow to $d \times d$.

5.8 Compression error ledger

For one archive capsule, let $M_1, \dots, M_u$ be exact incoming segment numerators and

$$M_u^\star = \sum_{j=1}^u M_j \quad (109)$$

the ideal retained numerator. At merge u , the implementation truncates

$$A_u = \widehat{M}_{u-1} + M_u, \quad \widehat{M}_u = P_r(A_u), \quad (110)$$

and reports

$$\delta_u = \|A_u - P_r(A_u)\|_F = \left(\sum_{j>r} \sigma_j(A_u)^2 \right)^{1/2}. \quad (111)$$

The slot field `error_bound` is updated linearly:

$$E_u = E_{u-1} + E_{\text{incoming}} + \delta_u. \quad (112)$$

For a freshly evicted exact live segment, $E_{\text{incoming}} = 0$.

Proposition 9 (Retained-capsule error bound) *In exact arithmetic, the ledger satisfies*

$$\|M_u^* - \widehat{M}_u\|_F \leq E_u. \quad (113)$$

Consequently, for any query q ,

$$\|(M_u^* - \widehat{M}_u)q\|_2 \leq E_u \|q\|_2. \quad (114)$$

Proof sketch. Write

$$M_u^* - \widehat{M}_u = (M_{u-1}^* - \widehat{M}_{u-1}) + (M_u - \widehat{M}_{\text{incoming}}) + (A_u - P_r(A_u)).$$

The triangle inequality and the inductive bounds on the first two terms yield (113) with the additive update (112). Submultiplicativity, $\|Eq\|_2 \leq \|E\|_F \|q\|_2$, yields (114). $\square$

Because z_i , route sums, sketch sums, and counts are accumulated separately from numerator compression, the ideal and compressed reads use the same denominator and router weights when compared at the same retained segmentation. For positive w_i ,

$$\|y^* - \widehat{y}\|_2 \leq \frac{\|q\|_2 \sum_{i \in \text{archive}} w_i E_i}{D_0}, \quad (115)$$

provided the mathematical denominator is $D_0 > 0$. This bound covers only currently retained capsules. It excludes overwritten history, finite-precision QR/SVD error, rounding from casting factors back to a lower dtype, and any task accuracy or recall guarantee.

5.9 Token-wise branch fusion and SwiGLU

Let $b_{t,j} \in \mathbb{R}^d$ denote active branch j 's output. Each branch has an independent RMS normalization and an unconstrained learned scalar:

$$\widetilde{b}_{t,j} = \beta_j \text{RMSNorm}_j(b_{t,j}), \quad \beta_j = \text{branch_scale}_j. \quad (116)$$

When more than one branch is active,

$$\alpha_t = \text{softmax}(W_f h_t + b_f), \quad (117)$$

$$f_t = \sum_j \alpha_{t,j} \widetilde{b}_{t,j}, \quad \alpha_{t,j} > 0, \quad \sum_j \alpha_{t,j} = 1. \quad (118)$$

With a single branch, the fusion projection is absent and the normalized, scaled branch is returned directly. For a multi-branch block, W_f, b_f are initialized to zero and every β_j to one, giving a uniform mixture at initialization.

Proposition 10 (Precise convexity statement) *For each token, f_t lies in the convex hull of the scaled normalized vectors $\{\widetilde{b}_{t,j}\}_j$. It is not generally constrained to the convex hull of the raw branch outputs $\{b_{t,j}\}_j$.*

Proof sketch. Softmax gives positive coefficients summing to one, proving the first claim. The second follows because each β_j is unconstrained and each branch has a distinct nonlinear RMS normalization. $\square$

The first residual update is

$$x'_t = x_t + \text{Dropout}(f_t). \quad (119)$$

Writing

$$(g_t^{\text{ffn}}, v_t^{\text{ffn}}) = \text{split}(W_{\text{in}} \text{RMSNorm}(x'_t)), \quad (120)$$

the feed-forward output and second residual are

$$\text{SwiGLU}(x'_t) = W_{\text{out}}^{\text{ffn}} \left[\text{SiLU}(g_t^{\text{ffn}}) \odot v_t^{\text{ffn}} \right], \quad (121)$$

$$x_{t,\text{out}} = x'_t + \text{Dropout}(\text{SwiGLU}(x'_t)). \quad (122)$$

5.10 Global causality and chunk/step equivalence

Let a lane state transition be written

$$(y_t, \mathcal{S}_t) = \mathcal{T}(x_t, \mathcal{S}_{t-1}).$$

The concrete $\mathcal{S}_t$ is the tuple of convolution histories, Gated Delta fast weights, local key–value windows, exact-cache tensors and metadata, and episodic current/live/archive state.

Proposition 11 (Inclusive causal block semantics) *With fixed parameters and incoming state, the output at token t depends only on the incoming state and tokens through t . No future token can change an earlier output. The current token can influence its own memory read in every memory lane.*

Proof sketch. The streaming stems are causal by (29). The Gated Delta lane updates F_t before reading it; local attention appends (k_t, v_t) before softmax; the exact cache decides insertion before reading; and the episodic lane writes and performs any seal, eviction, merge, or overwrite before aggregating its slots. Fusion and SwiGLU are token local. Composition over layers preserves causality. $\square$

The language-model loss uses `logits[:, :-1]` against `labels[:, 1:]`, so the inclusive memory read is compatible with next-token prediction.

Proposition 12 (Chunk and step equivalence) *Assume evaluation mode, fixed parameters, identical initial state, exact arithmetic, and the same token order. For any ordered partition of a sequence, concatenating chunk outputs yields the same outputs and terminal state as one full call or repeated one-token *step* calls.*

Proof sketch. Each forward method applies the same token transition in temporal order. Proposition 1 handles both convolutional stems. All other state updates are functions solely of the previous explicit state and current projected token. Associating the composition $\mathcal{T}_T \circ \dots \circ \mathcal{T}_1$ according to chunk boundaries does not change it. $\square$

Remark 3 (Finite precision and training mode) *The implementation tests chunk/step equality in evaluation mode with numerical tolerances, because vectorized linear projections and convolution kernels may have shape-dependent floating-point reduction order. In training mode, different partitioning can also consume dropout random numbers differently. Therefore Proposition 12 is not stated as bitwise equality for arbitrary training executions.*

Table 3: Per-token time and persistent state for fixed configuration.

Lane	Per-token time beyond or including stated projections	Persistent elements per batch item
Gated Delta	$\mathcal{O}(Hd_k d_v + \kappa_g d)$ recurrence/stem, plus its dense query/key/value/gate projections	$Hd_k d_v$ fp32 plus $(\kappa_g - 1)d$ model-dtype
Local exact	$\mathcal{O}(dW)$ attention plus QKV/output projections	$2dW$ model-dtype
Residual-surprise exact	$\mathcal{O}(C_x d)$ selection, full-shape update, and read, plus $\mathcal{O}(d^2)$ projections	$2C_x d$ model-dtype plus C_x priority and C_x valid entries
Episodic	Factor read, routing, sketching, dense projections, and amortized compression as in (130)	At most F_{epi} model-dtype elements

5.11 Persistent-state and compute bounds

Define the full conservative current/live slot size

$$F_{\text{live}} = 2Sd + d + d_r + Jm + 2, \quad (123)$$

and archive slot size

$$F_{\text{archive}} = 2rd + d + d_r + Jm + 2. \quad (124)$$

The terms are the two numerator factors, exact z , route sum, J sketch sums, count, and error ledger. The episodic state bound per batch item is

$$F_{\text{epi}} = (K + 1)F_{\text{live}} + CF_{\text{archive}} + (\kappa_e - 1)d. \quad (125)$$

This is conservative because a reachable current slot has fewer than S rows. The temporary fp32 QR/SVD workspace is not persistent state.

Combining all scheduled lanes gives the following logical tensor-byte bound per batch item for ordinary non-float64 execution:

$$\begin{aligned} \text{bytes}_{B=1} \leq & L [b_x(\kappa_g - 1)d + 4Hd_k d_v] + N_{\text{local}}(2b_x dW) \\ & + N_{\text{epi}}(b_x F_{\text{epi}}) + N_{\text{exact}}(2b_x C_x d + 5C_x). \end{aligned} \quad (126)$$

Batch size B multiplies the tensor payload by B . This accounting excludes parameters, optimizer state, autograd activations, QR/SVD workspace, Python metadata, allocator fragmentation, and excess backing-storage capacity. In particular, bounded inference state does not imply bounded training activation memory when the entire computation graph is retained.

Table 3 summarizes token costs. Dense projection costs are shown separately because the persistent-memory terms are the quantities that distinguish the four lanes.

At most $1 + K + C$ episodic slots participate in a read. Their factor-read cost is

$$\mathcal{O}(d[S(K + 1) + Cr]). \quad (127)$$

Route comparisons cost

$$\mathcal{O}((K + C + 1)(d_r + Jm)), \quad (128)$$

and the current FFT implementation constructs a token sketch in

$$\mathcal{O}(Jp[d_r + m \log m]). \quad (129)$$

Including amortized compression but suppressing dense projection constants,

$$\begin{aligned} \text{cost}_{\text{epi/token}} = \mathcal{O} \bigg(& d[S(K+1) + Cr] + (K+C+1)(d_r + Jm) \\ & + Jp[d_r + m \log m] + \frac{d(r+S)p_c + p_c^3}{S} \bigg). \end{aligned} \quad (130)$$

For a fixed configuration, recurrent-decode cost and persistent state are independent of total context length. A T -token scan nevertheless costs $\mathcal{O}(T)$ times the fixed per-token work. The current Python backend uses token loops, tuple concatenation, full-cache `torch.where`, and occasional QR/SVD; these asymptotic state bounds do not imply fused-kernel throughput.

5.12 Finite-capacity boundary

Every persistent lane is explicitly bounded:

- Gated Delta has $Hd_k d_v$ fp32 fast-weight elements;
- local attention stores only the last W projected key-value pairs;
- the residual-surprise cache stores at most C_x selected projected key-value pairs; and
- episodic memory stores one partial segment, at most K exact live segments, and at most C rank- r archive capsules.

For every capsule,

$$\text{rank}(\widehat{M}_i) \leq r. \quad (131)$$

Exact z_i , route, sketch, and count summaries cannot reconstruct numerator directions discarded by rank truncation.

Proposition 13 (Finite-state impossibility of arbitrary unbounded recall) *Fix model parameters and a persistent state representation with at most Q physical bits. Suppose query keys are supplied externally and each of N independent associated values belongs to an alphabet $\mathcal{V}$, with $|\mathcal{V}| \geq 2$. A system that recalls every possible assignment without error must satisfy*

$$Q \geq N \log_2 |\mathcal{V}|, \quad \text{equivalently} \quad N \leq \frac{Q}{\log_2 |\mathcal{V}|}. \quad (132)$$

Proof sketch. There are $|\mathcal{V}|^N$ possible independent value assignments. Perfect recall requires that assignments needing different answers not collapse to the same persistent state. A Q -bit state has at most 2^Q distinct physical encodings, so injectivity requires $2^Q \geq |\mathcal{V}|^N$. Taking base-two logarithms yields (132). $\square$

This counting bound is deliberately optimistic: it ignores floating-point noise, hash collisions, readout interference, rank limits, replacement, and optimization error. The implemented consequences are concrete. Exact-cache records outside the stable top- C_x set are absent from that lane; local-attention records older than W are absent from that lane; an overwritten episodic capsule is absent entirely; and retained archive numerators are generally lossy. Increasing A extends (95) without adding archive slots, but forces more segments through the same rank- r capsule and can only increase the opportunity for approximation and interference. No equation in this section supports exact recall over an unbounded stream.

5.13 Code correspondence and validation boundary

The equations above were cross-checked against the following implementation sites:

- `gated_delta.py`: decay, erase/write recurrence, RMS surprise, fp32 fast weights, and write-then-read order;
- `local_attention.py`: append/truncate/read order, learned recency slope, fp32 softmax, and W -bounded state;

- `exact_cache.py`: detached strict replacement, batch-local first-empty/minimum selection, exact softmax read, and mixed-dtype state accounting;
- `episodic.py`: positive feature map, separate M and z , TensorSketch routing, stabilized mass normalization, ring lifecycle, retained horizon, and state formula;
- `compression.py`: reduced QR, small-core SVD, symmetric singular split, discarded tail, and additive error ledger;
- `model.py`: parallel branch execution, token-wise softmax fusion, residual ordering, and SwiGLU; and
- `state.py`: the exact persistent tensor fields counted in (126).

The corresponding tests check Gated Delta scan/step equality and fixed state; local scan/step equality, window capping, and causality; exact-cache top-priority retention, strict ties, scan/step equality, causality, dtype handling, and constant allocation; episodic denominator separation, conditional routing mass conservation, chunk equivalence through archive eviction, causality, and bounded state; dense-versus-small-core compression and the error ledger; and full-model chunk/step equality in evaluation mode. The shipped release record `results/release_validation.json` reports fifty passing tests for the audited snapshot.

Two measured artifacts are useful only as implementation anchors, not as theorems. The protocol `aluclu_diagnostic_next_token_mqar_v1` records 90.43% answer accuracy and 9,200 bytes of batch-one state for its small learned CPU diagnostic. The protocol `aluclu_reference_performance_v2` records 164,576 bytes after a saturated batch-one stream, a required warmup horizon of 528 tokens, and a median 143.71 recurrent tokens/s in its reported single-thread CPU environment. These values establish that the specified mechanics executed under those exact configurations; they do not prove large-scale language-model quality, hardware-independent speed, or superiority over another architecture.

6 Reference Implementation

6.1 Package layout

The MIT-licensed reference implementation targets Python 3.10 or later and PyTorch 2.10 or later. Table 4 maps the specification to the released modules.

Table 4: Implementation map.

Module	Responsibility
<code>config.py</code>	Validated immutable configuration objects and branch schedules.
<code>state.py</code>	External state dataclasses, detach utilities, and logical tensor-payload accounting.
<code>gated_delta.py</code>	Causal stem, Gated Delta Rule-2 token recurrence, write-surprise signal, sequence and step paths.
<code>local_attention.py</code>	Rolling exact local KV state and causal softmax read.
<code>exact_cache.py</code>	Deterministic fixed-capacity priority insertion and exact projected-KV attention.
<code>episodic.py</code>	Current/live/archive life cycle, slot aggregation, router, and archive ring.
<code>compression.py</code>	Thin-QR small-core SVD factor truncation and discarded-tail accounting.
<code>sketch.py</code>	Reproducible signed CountSketch maps and TensorSketch features.

Module	Responsibility
<code>model.py</code>	Block scheduling, branch fusion, residual path, language model head, loss, and recurrent API.
<code>mqr.py</code>	Small ALUCLU diagnostic MQR generator and metric.
<code>zoology_mqr.py</code>	Historically aligned Zoology MQR semantics and masked loss.
<code>protocols/*.json</code>	Immutable protocol manifests, including the historical mismatch record and a separately identified repair.

6.2 Configuration validation

External dimensions and capacity budgets are validated before tensor allocation. The configuration rejects non-positive dimensions, incompatible head splits, invalid feature bounds, even sketch-replica counts where a median requires an odd count, and inconsistent scheduling values. These checks are not cosmetic: a silently truncated head dimension or non-positive feature floor would invalidate the equations in [Section 5](#).

The implementation intentionally does not force $r + S < d$. Configurations with $r + S \geq d$ remain mathematically valid, but the small SVD core can grow to $d \times d$, losing the intended compression-time advantage. This is reported as a performance condition rather than prohibited as a semantic error.

6.3 Numerical precision

The recurrent fast-weight matrix, token-dependent decay, and Gated Delta update execute in fp32 even when the surrounding model uses fp16 or bfloat16. Local and exact-cache softmax probabilities are computed in fp32 before being cast back to the query dtype. QR and SVD promote fp16 or bfloat16 inputs to fp32, then cast retained factors and the reported discarded tail back to the input dtype.

These promotions protect the most sensitive reductions, but they do not make the whole architecture numerically exact. Episodic persistent factors, z , route sums, sketches, and the error ledger remain in model dtype. Furthermore, the ledger covers algebraic rank-truncation tails; it does not add an explicit term for floating-point QR/SVD error or down-cast rounding.

6.4 State ownership and graph control

State dataclasses are treated as values at the public boundary. Each call returns a new logical state rather than mutating a hidden global cache. This supports interleaved streams and makes state transfer visible to callers. It also means the caller owns lifecycle decisions:

- pass `None` to begin a new stream;
- carry returned state to continue a stream;
- call `model.detach_state(state)` at a training boundary when the old autograd graph must be released;
- do not reuse one sample’s state for an unrelated sample.

The v0.1.0 API does not include a per-example reset mask for ragged batches and does not define a versioned state serializer. Production serving would need both before arbitrary stream regrouping or durable cache persistence.

6.5 Determinism and discrete cache selection

Exact-cache priority selection is batch-local and deterministic for a fixed input. Empty slots are filled in index order. Once full, the first minimum selected by the tensor reduction is replaced

only if the incoming priority is strictly greater. Equal priorities preserve the older record.

The priority is detached before discrete selection. Therefore cache-read loss does not differentiate through the ranking decision into the Gated Delta write residual. Accepted key/value projections can receive gradient through the read; rejected records do not receive a key/value write contribution from that step. This boundary is an intentional implementation fact, not a claim that hard selection is optimal.

6.6 Protocol provenance

MQAR is particularly sensitive to target placement and next-token shifting. The repository therefore assigns distinct protocol identifiers to:

1. the small ALUCLU next-token diagnostic generator;
2. the target semantics reconstructed from the historical Zoology generator;
3. the historical raw manifest with its documented layer/config-count mismatch;
4. the smallest runnable repair, under a new identity.

This prevents a repaired configuration from silently inheriting the identity of a non-runnable historical command and prevents two different target alignments from being reported as the same benchmark.

6.7 Testing strategy

The 50-test suite emphasizes invariants that can detect semantic drift:

- dense equivalence of factor compression at sufficient rank;
- agreement between discarded singular tail and best-rank Frobenius error in float64 tests;
- error-ledger accumulation across repeated merges;
- exact z denominator preservation after factor compression;
- router identity at zero temperature and denominator-mass conservation;
- local, episodic, and exact-cache causality;
- full scan, multiple chunkings, and token-step equivalence;
- strict top- C_x cache retention and fixed initialized cache shape;
- state saturation after sufficient streaming tokens;
- archive capacity, compression depth, and ring eviction behavior;
- finite outputs, losses, and gradients on small random examples.

Tests establish implementation invariants, not model intelligence. In particular, a passing cache-retention test proves that the policy keeps the largest priorities, not that priority predicts future utility.

6.8 Reference-backend performance characteristics

The current backend uses a Python loop over tokens. Exact-cache updates employ full-shape `torch.where`; episodic transitions create tuples and concatenate factors; archive boundaries invoke library QR/SVD routines. These choices favor clarity and direct comparison with the specification. They should be treated as a correctness oracle and CPU fallback.

A production accelerator path should preserve the same externally tested semantics while introducing:

1. chunk-parallel or scan kernels for Gated Delta recurrence;
2. fused causal stems, projections, recurrence, and output gating;
3. ring-buffer local KV storage without repeated concatenation;
4. in-place indexed cache writes with a compact selection reduction;
5. packed episodic tensors with explicit masks instead of Python tuples;
6. asynchronous or scheduled capsule compression;
7. profiler-guided fusion constrained by numerical parity tests.

Kernelization is a separate engineering phase because a fast kernel that changes replacement ties, write-then-read order, fp32 accumulation, or segment-boundary transitions would implement a different model.

6.9 Minimal recurrent use

Listing 1: One-token recurrent inference.

```
import torch
from aluclu import AlucluLanguageModel, build_research_config

config = build_research_config(
    d_model=64,
    n_layers=4,
    local_window=64,
    segment_size=16,
    live_segments=16,
    archive_slots=4,
    archive_segments_per_slot=4,
    archive_rank=8,
    exact_cache_capacity=64,
)
model = AlucluLanguageModel(vocab_size=8192, config=config).eval()

state = None
token = torch.tensor([7])
logits, state = model.step(token, state)
```

The explicit `state` variable in [Listing 1](#) is essential: dropping it converts every call into a one-token independent sequence.

7 Evaluation Methodology

7.1 Two non-interchangeable MQAR protocols

Multi-query associative recall (MQAR) reveals whether a model can bind keys to values and retrieve several bindings later in a sequence. It is also easy to mis-measure by shifting targets twice or comparing different generators. ALUCLU therefore ships two separately identified protocols.

Table 5: MQAR paths in the reference implementation.

Property	Diagnostic next-token MQAR	Zoology-aligned MQAR
Implementation	<code>generate_mqar_batch</code> , <code>mqar_accuracy</code>	<code>generate_zoology_mqar_batch</code> , <code>zoology_mqar_loss/accuracy</code>
Target meaning	Predict the value after a query; normal LM shifting applies	Targets are already placed at the query position; no extra shift is allowed
Purpose	Compact developer learnability and regression diagnostic	Protocol-compatible study of historical Zoology data semantics
Claim ceiling	Local evidence for the exact declared distribution	Comparable only after full dataset, grid, seed, and selection policy are run

The historical raw manifest records vocabulary 8192, training and test sizes 100,000 and 3,000, data seed zero, test-seed offset 10, power parameter 0.01, model seed 123, weight decay 0.1, and up to 64 epochs. It records four (length, pairs, batch) points: (64, 4, 512), (128, 8, 512), (256, 16, 256), and (512, 64, 128). A mismatch between a historical four-layer description and

two configuration entries is preserved in the raw manifest. The repository’s smallest runnable interpretation has a different protocol identity; it is not silently presented as the original experiment.

7.2 P0: correctness and small-scale decision gate

The tests cover deterministic invariants, but the architecture should not scale merely because those tests pass. The proposed P0 matrix in Table 6 determines whether each lane earns its state and latency cost.

Table 6: Proposed P0 experiments. These sweeps are protocols, not current results.

Experiment	Variants and sweep	Metrics	Decision rule
Partition equivalence	All lanes and full model; token, prime-sized, random, adversarial chunks	Maximum/mean error in output, state, and gradients	Remain within dtype-specific preregistered tolerance
Causal prefix	Replace future suffixes across seeds	Maximum error on unchanged prefix	No future-dependent change beyond tolerance
State saturation	Streams below and far beyond retained horizon	Logical bytes, allocated/reserved memory, post-saturation slope	Logical persistent slope equals zero after allocation
Archive wrap	Multiple ring wraps and rank sweep	Counts, cursor, denominator minimum, reconstruction error, ledger	No bound violation; ledger never under-reports algebraic tail
Cache semantics	Increasing, decreasing, tied, outlier, duplicate, heterogeneous priorities	Retained K/V, priorities, scan/step outputs	Strict greater-than, batch-local policy matches oracle
Recall curves	Lane subsets across distance, pair count, overwrites, capacity ratios	Accuracy/NLL by distance and $T/H_{\max}$	Complete seed distributions and curves, not selected points
Numerical stress	fp32/bfloat16, long carried state, repeated compression	NaN/Inf, norms, denominator, ledger, first failure index	All declared paths finite or failure fully localized
CPU reference	Batch 1/4/16, short and saturated streams	Token/s, p50/p95/p99 latency, logical/resident memory	Descriptive cost curves only

Quality comparisons should use at least five seeds and retain all runs. Accuracy differences should use paired bootstrap intervals or an appropriate interval for paired proportions. Scale-up is justified only if the full model or a simpler ablation is nondominated on a quality–state–latency frontier. A lane that supplies no reproducible Pareto point should be removed rather than retained for visual symmetry.

7.3 P1: model quality and long-context transfer

P1 begins with a matched-token 100–200M-parameter pilot. A 1B-class run is permitted only if the pilot effect survives multiple seeds. This staged design reduces the risk of spending scale

to conceal an architectural weakness.

7.4 Baseline contract

The minimum rerun baseline set is:

1. a causal Transformer with a mature exact-attention kernel and a fixed-window Transformer;
2. Based [Arora et al., 2024b];
3. pure Gated Delta and Gated DeltaNet-2 [Yang et al., 2025, Hatamizadeh et al., 2026];
4. Mamba and current maintained Mamba-family controls [Gu and Dao, 2024, Dao and Gu, 2024, Lahoti et al., 2026];
5. a reproducible residual-selected bounded-cache control, including HOLA if its released protocol and implementation are suitable [Cui, 2026];
6. full ALUCLU and every principal lane subset.

External paper numbers provide context but do not enter a comparative result table unless rerun under the shared data, token, parameter, precision, and hardware contract. A baseline with growing global KV state may be a valuable quality control, but must not be labeled context-independent.

7.5 Ablation program

Table 8: Minimum proposed ablations.

Ablation	Hypothesis isolated	Primary measurements
Gated Delta only	Dense recurrent baseline	NLL, recall by distance, state, throughput
Local only; recurrent + local	Exact recency and complementarity	Near/far recall, state, latency
Episodic combinations	Exact segments and compressed archives	Recall around eviction, ledger, compression-tail latency
Exact-cache combinations	Residual-selected events versus recency	Recall by retained age, turnover, state and read cost
Archive off; K, C, A sweep	Exact-live versus compressed allocation	Capacity-boundary recall, error and bytes
Routing off; degree one; sketch router	Incremental value of routing and polynomial features	Quality, wall time, bytes, calibration
p, m, J sweep	Sketch collision and compute budget	Recall, score variance, FFT cost, stability
Rank r sweep	Compression capacity	Dense reconstruction error, ledger, p99 latency
Window W and cache C_x sweeps	Exact memory allocation	Age curves, bytes, latency, traffic
Uniform, fixed, learned fusion	Token-conditional lane selection	Quality, entropy, branch collapse, gradients
Residual, FIFO, recency, random, reservoir admission	Specific value of write-residual priority	Recall, age, diversity, turnover, runtime

Ablation	Hypothesis isolated	Primary measurements
Priority aging and diversity controls	Cache freezing and duplicate retention	New-information recall and unique clusters
fp32 versus bfloat16 state	Precision versus traffic	Drift, finite values, bytes and quality

7.6 Systems scenarios

Reference code and optimized kernels must be reported separately. The proposed systems matrix includes interactive short prompts, interactive 16K–128K prompts, batched throughput, saturated streaming across multiple archive wraps, prefill scaling, and decode-only runs from saturated snapshots. Required metrics include time to first token, p50/p95/p99 time per output token, aggregate token/s, logical and allocated state, peak resident memory, DRAM bytes/token, and joules/token.

Archive-boundary tokens must be tagged separately because QR/SVD can create a tail-latency mode that the median hides. Random input generation, compilation, autotuning, model loading, and state construction are reported as separate regions. Warm-up, device synchronization, clock policy, and raw samples are preserved.

7.7 Fused-kernel correctness gate

No fused CUDA or Triton implementation is part of the current evidence. A future accelerator path must agree with the recurrent oracle on:

- forward outputs and every carried-state tensor;
- input and parameter gradients;
- fp32, bfloat16, and declared mixed-precision tolerances;
- masking, variable lengths, random chunking, and state carry;
- cache ties/replacement, segment sealing, live eviction, archive merge, and ring wrap;
- numerical stress without silent NaN, Inf, or denominator failure.

Only after this gate should kernel profiling drive fusion: recurrence scan, production sliding-window attention, indexed cache insertion/read, packed episodic reads and routing, and finally compression scheduling.

7.8 Memory traffic and energy

DRAM bytes/token must come from hardware counters rather than shape multiplication. Reports should include DRAM reads/writes, cache hit rates, achieved bandwidth, occupancy, matrix-unit utilization, launch counts, and the time share of FFT, QR, and SVD operations.

For sampled device power $P(t)$, idle power P_{idle} , and N generated tokens, define

$$E_{\text{token}} = \frac{1}{N} \int_{t_0}^{t_1} \max(P(t) - P_{\text{idle}}, 0) dt. \quad (133)$$

The measurement must report sampling interval, integration rule, idle procedure, power/clock policy, workload duration, and uncertainty. Short workloads are repeated long enough to exceed sensor resolution.

7.9 Result-artifact schema

Every future result artifact should contain:

- a versioned protocol ID and source commit or archive hash;
- full model, data, optimizer, schedule, precision, and seed configuration;

- raw per-seed and per-repeat samples;
- software, hardware, driver/runtime, thread count, and kernel mode;
- parameter count, logical state bytes, allocated peak, and saturation test;
- explicit classification as smoke, diagnostic, model-only, or end-to-end evidence.

The shipped CLIs write JSON atomically, reducing the chance that an interrupted process leaves a partial result that looks complete.

8 Current Measured Evidence

8.1 Evidence boundary

This section contains every numerical result used to characterize v0.1.0. There is no hidden large-model result and no accelerator benchmark. The measurements fall into three categories: invariant tests, a small learnability diagnostic, and a saturated-state CPU timing run.

8.2 Automated release gate

The recorded release gate ran on Windows, Python 3.11.15, PyTorch 2.13.0+cpu. It completed bytecode compilation, 50 tests, two training smoke runs, and one performance smoke run with zero non-zero return codes.

The two-step and one-epoch smoke tasks report zero evaluation accuracy on their tiny held-out samples. That is expected to be possible after almost no optimization and is preserved in the JSON rather than filtered out. The release criterion for those jobs is numerical and operational completion, not task mastery.

8.3 Small MQAR learnability diagnostic

A separate diagnostic trained a 14,288-parameter, two-layer model for 300 steps on the ALUCLU-specific next-token MQAR generator. The setup used $d = 16$, batch size 32, two key-value pairs, two queries, four key classes, four value classes, learning rate 3×10^{-3} , and seed zero. Sixteen evaluation batches produced:

$$\text{answer accuracy} = 0.904296875, \quad (134)$$

$$\text{evaluation loss} = 0.1969321771, \quad (135)$$

$$\text{elapsed time} = 29.9342 \text{ s}, \quad (136)$$

$$\text{training throughput} = 2886.33 \text{ tokens/s}, \quad (137)$$

$$\text{batch-1 terminal state} = 9200 \text{ bytes}. \quad (138)$$

This is evidence that the integrated model can learn one deliberately easy associative-recall distribution. It is not an estimate of long-context capacity: the vocabularies are tiny, only one seed was run, the relevant sequence lengths are short, and no baseline comparison is attached.

8.4 Saturated-state CPU reference timing

The reference timing configuration used $d = 32$, four layers, vocabulary 8192, 363,385 parameters, one PyTorch CPU thread, 128-token prefill, 560 warm-up tokens, and three repeated 32-token decode measurements. The 560-token warm-up exceeds the reported 528-token state-saturation requirement, so decode started with every configured episodic capacity allocated.

The saturated logical state payload for batch one was 164,576 bytes. The wide range across only three samples illustrates why this result cannot support a systems-leadership claim. The benchmark is nevertheless useful as a regression anchor because it records thread count, environment, model configuration, timed-region policy, samples, and saturation status.

8.5 What can be concluded

The combined evidence supports four statements.

1. The released code paths compile and execute in the recorded CPU environment.
2. The tested causal, state, compression, routing, and equivalence invariants hold for the test domain.
3. The full four-lane model is trainable on a small diagnostic and can exceed chance substantially under that protocol.
4. A fully allocated recurrent state can decode indefinitely without persistent tensor payload growing with the number of processed tokens.

It does not yet answer whether the lane allocation is Pareto efficient, whether the surprise policy predicts future utility, whether the archive improves language modeling, whether training remains stable at scale, or whether a specialized implementation outperforms optimized attention or state-space baselines.

9 Limitations, Failure Modes, and Risk Controls

9.1 Memory is bounded because information is lost

Every lane has an explicit loss mechanism. Gated Delta mixes information into a finite fast-weight matrix and applies learned decay. Local attention drops tokens older than W . The exact cache rejects all but C_x priorities. The episodic hierarchy truncates archive numerators to rank r and eventually overwrites capsules. Bounded state is therefore not a free optimization; it is a controlled forgetting policy.

9.2 Archive approximation

When the singular tail does not fit rank r , the archive loses numerator directions. The error ledger makes a conservative algebraic bound visible, but the bound is not a recall guarantee and can become loose after many merges. It excludes expired capsules and does not explicitly include finite-precision factorization error. Exact denominator accumulation can also create a semantic asymmetry: the normalization mass still represents all retained keys while the numerator may no longer represent all corresponding value directions.

9.3 Abrupt ring eviction

Once the archive ring is full, advancing to a new capsule overwrites an entire old capsule. The retained age horizon therefore changes in a sawtooth pattern rather than one token at a time. Increasing the number of segments A per capsule extends nominal retention without adding capsule state, but also makes each overwrite larger and forces more heterogeneous information into the same rank.

9.4 Router error and conditioning

Finite TensorSketch features can collide. The degree-one and polynomial scores can prefer the wrong slot, and the learned bounded temperature can be negative, inverting the score preference. Mass normalization preserves a global denominator quantity under its stated arithmetic conditions; it does not make routing semantically correct. Large log-weight bounds can amplify a wrong slot and worsen numerical conditioning.

9.5 Surprise is not future utility

The exact cache ranks tokens by the RMS residual of the Gated Delta write target. A large residual means the recurrent lane had difficulty writing that token under its current state and

parameters. It does not imply that a future query will need the token. Conversely, a low-residual token can be crucial later. Because priority has no age decay, early outliers can occupy the cache for a long time. The local lane provides recency independently, but does not solve global cache-selection mismatch.

9.6 Hard selection limits learning

The detached priority and discrete replacement index block end-to-end credit from cache utility into the surprise ranking. This avoids unstable gradients through sorting and makes semantics auditable, but may leave performance on the table. Differentiable relaxations, learned admission heads, or straight-through estimators are possible research variants; they must be evaluated as different models and should not replace the deterministic oracle without evidence.

9.7 Training and numerical stability

Long recurrent scans retain autograd history unless callers detach state or use checkpointing/truncated backpropagation. QR/SVD gradients can be unstable near repeated singular values. fp32 promotion protects selected operations but does not prevent all mixed-precision drift. The current tests cover small random gradients, not large-scale distributed training, optimizer-state sharding, quantization, or adversarial numerical cases.

9.8 Serving gaps

The public API lacks per-example reset masks for continuously regrouped batches, a versioned state serialization format, schema migration, and a production cache allocator. Python tuples and per-token loops are unsuitable for high-throughput serving. State-byte accounting measures logical tensor payload, not allocator fragmentation, reserved device memory, or temporary factorization workspace.

9.9 Evaluation gaps

The available learnability result uses one seed and one small synthetic distribution. There is no matched pretraining, no perplexity result, no RULER or HELMET evaluation, no multi-GPU training study, no energy measurement, no quality-matched throughput comparison, and no fused accelerator kernel. Accordingly, this release cannot support claims of state of the art, production readiness, universal efficiency, or the “world’s fastest” architecture.

9.10 Risk register

Table 11: Principal engineering risks and observable controls.

Risk	Mechanism	Current control or measurement
Archive information loss	Singular tail exceeds rank budget	<code>error_bound</code> , rank and A sweeps
Temporal cliff	Capsule ring overwrite	Archive cursor, segment count, retained-horizon accounting
Wrong slot amplification	Router collision or temperature sign	Positive bounded weights, mass check, router ablations
Cache freezing	No priority aging	Priority histogram, retention age, local-lane comparison

Risk	Mechanism	Current control or measurement
Missed critical token	Residual is not utility	Oracle admission and random/recency baselines
Nondifferentiable admission	Detached priority and hard index	Explicit gradient-boundary tests
Factorization instability	Repeated singular values	fp32 promotion, finite-gradient tests, conditioning metrics
Mixed-precision drift	Low-precision persistent statistics	fp32 oracle comparison and dtype sweeps
State leakage between streams	Missing reset mask	Explicit caller-owned state; serving feature required
Misleading speed result	Python reference vs fused baseline	Backend label, timed-region manifest, no superiority claim

9.11 Responsible claim policy

Future reports should attach each headline to a protocol ID and evidence file. “Bounded state” must name the configuration and payload definition. “Exact” must identify which selected projected records are exact. “Long context” must report the age distribution and approximation regime. “Faster” must name hardware, precision, batch, sequence lengths, compiler, kernel, and achieved quality. This policy is part of the architecture because an optimization that cannot be measured honestly cannot be engineered reliably.

10 Conclusion

ALUCLU is an attempt to make memory trade-offs architectural objects rather than footnotes. Dense recurrence, exact recency, sparse exceptional records, and compressed episodes are assigned separate bounded states, update rules, and failure modes. Learned fusion lets the model combine those states at each token, while the public recurrent interface keeps state ownership explicit.

The reference implementation establishes a precise baseline for the idea. It defines inclusive write-then-read causality, deterministic cache replacement, an auditable episodic life cycle, small-core factor compression, a compression-error ledger, conditional mass conservation, and concrete persistent-byte formulas. The automated tests and small CPU measurements show that these mechanisms are executable and internally consistent within the tested domain.

The work is intentionally unfinished at the level where strong empirical claims begin. The central research questions now have measurable forms: whether residual surprise selects useful long-range evidence, whether the episodic lane improves quality per state byte, whether token-wise fusion specializes by dependency type, and whether faithful fused kernels move the quality-throughput-memory Pareto frontier. The evaluation program in [Section 7](#) is designed to answer those questions without confusing mechanical correctness with model quality or reference code with optimized systems.

The practical thesis is simple:

Layer the context, bound the memory, expose the loss, and measure the result.

If later experiments reject a lane or a routing rule, the architecture’s modularity makes that a useful result rather than an untraceable failure. If they validate the composition, the same specification provides the semantic oracle required to engineer it aggressively without losing correctness.

A Notation and Configuration Crosswalk

Table 12: Mathematical notation and public configuration names.

Symbol	Configuration field	Meaning	Primary effect
B	<code>batch_size</code>	Batch size	All persistent tensor payloads scale linearly
T	<code>call input length</code>	Tokens processed in one call	Scan work and training activations
d	<code>d_model</code>	Model width	Projection, FFN, and every lane
L	<code>n_layers</code>	Block count	Parameters, compute, recurrent state
H	Gated Delta <code>n_heads</code>	Recurrent head count	Fast-weight partition
d_k	<code>head_key_dim</code>	Recurrent key width	Fast-weight rows
d_v	<code>head_value_dim</code>	Recurrent value width	Fast-weight columns
W	<code>local_window</code>	Exact recent window	$2dW$ state per local lane
C_x	<code>exact-cache capacity</code>	Selected projected KV records	Cache state and read cost
S	<code>segment_size</code>	Tokens per episodic segment	Exact factors and seal frequency
K	<code>live_segments</code>	Exact completed segments	Recent exact factor budget
C	<code>archive_slots</code>	Compressed capsules	Temporal archive granularity
A	<code>archive_segments_per_slot</code>	Segments per capsule	Retention versus compression depth
r	<code>archive_rank</code>	Retained numerator rank	Archive error, state, read, and SVD cost
d_r	<code>router_dim</code>	Route-vector width	Route state and degree-one score
p	<code>router_degree</code>	Polynomial degree	TensorSketch cost and similarity sharpness
m	<code>sketch_dim</code>	Sketch width	Collision/variance budget
J	<code>n_sketches</code>	Independent sketches	Median robustness and cost
κ_g	Gated Delta <code>conv_kernel</code>	Recurrent stem width	Short causal filtering
κ_e	<code>episodic conv_kernel</code>	Episodic stem width	Memory write preprocessing

B Persistent State Shapes

For one scheduled block and batch B , the logical tensors are:

Table 13: State tensors and semantics.

Field	Shape	Meaning
Gated Delta history	$B \times (\kappa_g - 1) \times d$	Causal depthwise convolution context
Gated Delta fast weight	$B \times H \times d_k \times d_v$	fp32 recurrent associative matrix
Local key/value	each $B \times H_l \times W \times (d/H_l)$	Last W projected records
Exact-cache key/value	each $B \times H_x \times C_x \times (d/H_x)$	Selected projected records
Exact-cache priority	$B \times C_x$	Detached write-residual RMS
Exact-cache valid	$B \times C_x$	Occupancy mask
Episodic current/live factor	each side $B \times S \times d$ per slot	Exact numerator factor rows
Archive factor	each side $B \times r \times d$ per capsule	Truncated numerator factors
Slot denominator	$B \times d$	Exact sum of positive keys
Slot route sum	$B \times d_r$	Sum of normalized route vectors
Slot sketch sum	$B \times J \times m$	Sum of TensorSketch features
Slot count/error	each B	Token count and Frobenius error ledger

The archive/current/live tuples carry Python metadata in addition to tensors. The byte formulas in the main text count logical tensor payload and exclude those objects, allocator bookkeeping, backing-storage slack, and temporary workspaces.

C Operational Pseudocode

C.1 One block step

Listing 2: Semantic order of a single block step.

```

function ALUCLU_BLOCK_STEP(x, state):
    h = RMSNorm_mix(x)

    gdn_out, surprise, gdn_state =
        GATED_DELTA_WRITE_THEN_READ(h, state.gdn)

    branches = [gdn_out]
    if local lane exists:
        local_out, local_state =
            LOCAL_APPEND_THEN_ATTEND(h, state.local)
        branches.append(local_out)

    if episodic lane exists:
        epi_out, epi_state =
            EPISODIC_WRITE_TRANSITION_THEN_READ(h, state.episodic)
        branches.append(epi_out)

    if exact-cache lane exists:
        cache_out, cache_state =
            CACHE_SELECT_WRITE_THEN_ATTEND(h, surprise, state.cache)
        branches.append(cache_out)

```

```

normalized = [
    scale[j] * RMSNorm_branch[j](branches[j])
    for j in active_branches
]
alpha = softmax(fusion_projection(h))
mixed = weighted_sum(alpha, normalized)

x1 = x + dropout(mixed)
x2 = x1 + dropout(SwiGLU(RMSNorm_ffn(x1)))
return x2, assembled_new_state

```

C.2 Exact-cache insertion

Listing 3: Deterministic strict replacement for one batch item.

```

function INSERT(cache, key, value, incoming_priority):
    p = stop_gradient(incoming_priority)
    if any slot is invalid:
        j = first invalid slot
        should_write = true
    else:
        j = argmin(cache.priority)
        should_write = p > cache.priority[j]

    if should_write:
        cache.key[:, j, :] = key
        cache.value[:, j, :] = value
        cache.priority[j] = p
        cache.valid[j] = true
    return cache

```

C.3 Episodic transition

Listing 4: Current, live, and archive life cycle.

```

function WRITE_EPISODIC(token_slot, state):
    state.current = concatenate_factors(state.current, token_slot)
    if state.current.factor_count < segment_size:
        return state

    sealed = state.current
    state.current = empty
    append sealed to state.live

    if len(state.live) <= live_segments:
        return state

    oldest = pop_front(state.live)
    if archive_slots == 0:
        return state

    if active capsule has fewer than A segments:
        active = TRUNCATE_TO_RANK(active + oldest)
    else:
        advance ring cursor

```

```

    active = TRUNCATE_TO_RANK(oldest)
    update capsule segment count
    return state

```

D Reproduction Commands

D.1 Environment and editable install

```

python -m venv .venv
python -m pip install --upgrade pip
python -m pip install -e ".[test,lint]"

```

On Windows PowerShell, use `.venv\Scripts\python.exe`; on Linux or macOS use `.venv/bin/python`.

D.2 Complete portable release gate

```
python scripts/validate_release.py
```

The gate compiles source, runs tests, executes both training smoke paths, runs the timing smoke path, hashes the declared source set, and writes `results/release_validation.json`.

D.3 Recorded diagnostic

```

python scripts/train_mqar.py \
  --steps 300 --batch-size 32 \
  --n-pairs 2 --n-queries 2 \
  --key-vocab-size 4 --value-vocab-size 4 \
  --d-model 16 --n-layers 2 \
  --learning-rate 0.003 --eval-batches 16 \
  --device cpu \
  --output results/diagnostic_learned_cpu.json

```

D.4 Recorded saturated reference benchmark

```

python scripts/benchmark_stream.py \
  --d-model 32 --n-layers 4 --vocab-size 8192 \
  --warmup-tokens 560 --measure-tokens 32 \
  --prefill-tokens 128 --repeats 3 \
  --torch-threads 1 --device cpu \
  --output results/reference_cpu_saturated.json

```

The benchmark excludes random-token generation from timed regions and records whether the required state horizon was reached before decode.

E Test-to-Claim Traceability

Table 14: Representative invariant families and the claims they constrain.

Invariant family	Primary test module	Claim constrained
Thin-core dense equivalence and tail error	<code>test_compression.py</code>	Compression implements the specified factor product and error metric
Exact denominator and routing mass	<code>test_episodic.py</code>	Archive z is independent of SVD factors; router normalization follows its contract
Episodic segmentation and eviction	<code>test_episodic.py</code>	Current/live/archive counts remain bounded and transitions are causal
Gated Delta scan/step equality	<code>test_recurrent_and_local.py</code>	Recurrent API matches full evaluation scan
Local horizon and causality	<code>test_recurrent_and_local.py</code>	Local state retains at most W records and does not read future tokens
Strict top-priority retention	<code>test_exact_cache.py</code>	Cache policy keeps the configured number of highest distinct priorities
Exact-cache shape saturation	<code>test_exact_cache.py</code>	Cache allocation is length independent after initialization
Hybrid full/chunk/step equality	<code>test_model.py</code>	Integrated lane ordering and state carriage agree
Zoology target placement	<code>test_zoology_mqar.py</code>	Same-position labels are not shifted a second time

F Suggested Citation and License

```
@software{aluclu2026,
  author = {Kaan Kadir Aluclu},
  title = {ALUCLU: Adaptive Layered Unified Context
           with Learned Updates},
  year = {2026},
  version = {0.1.0},
  url = {https://github.com/kaannsaydamm/ALUCLU},
  license = {MIT}
}
```

The source code, paper source, and distributed artifacts are released under the MIT License. The spelling in machine-readable ASCII identifiers is ALUCLU; the author’s family name is Aluclu.

G Release Checklist

The following checklist is used before publishing a new evidence-bearing revision:

1. no placeholder implementation or unresolved document marker remains;
2. formatter, linter, bytecode compilation, and all tests pass;
3. every benchmark records protocol ID, full configuration, environment, samples, and output path;

4. state saturation is reported rather than inferred;
5. generated distributions contain package data and MIT metadata;
6. multilingual README files agree on architecture and license;
7. the paper compiles without undefined references or citations;
8. rendered PDF pages are visually inspected for clipping and overlap;
9. source archives and user-facing files have SHA-256 checksums;
10. the Git commit and pull request describe measured evidence without extending its claim scope.

Table 7: Proposed model-quality matrix.

Evaluation	Question	Sweep	Required report
Held-out text NLL	Does the memory design improve language modeling?	Training length and controlled extrapolation	Token-weighted NLL, perplexity, intervals, all seeds
Zoology MQAR	Where does associative recall fail relative to budgets?	Full grid, learning rates, distances, overwrites, capacity ratios	Accuracy by point and $T/H_{\max}$, protocol identity
RULER [Hsieh et al., 2024]	Can the model retrieve, trace, and aggregate?	4K through 128K where supported	Task-by-length curves, not only macro average
NoLiMa [Modarressi et al., 2025]	Does retrieval survive weak lexical overlap?	Supported lengths, stratified overlap	Absolute score and degradation from short-context score
HELMET [Yen et al., 2025]	Does synthetic recall transfer to application tasks?	All supported categories and lengths	Per-category uncertainty; synthetic kept separate
BABILong [Kurato et al., 2024]	Does carried memory support multi-step reasoning?	Supported long lengths and declared fine-tuning	Task scores, training regime, confidence intervals

Table 9: Recorded release-gate outcomes. Smoke runs check executable paths; their accuracy is not a trained-model result.

Gate	Elapsed (s)	Return code	Interpretation
Compile source, scripts, tests	0.152	0	Syntax/import compilation
Pytest suite	5.815	0	50 tests passed
Diagnostic MQAR smoke	3.368	0	Two optimizer steps completed
Zoology-aligned MQAR smoke	3.164	0	One tiny epoch completed
Stream benchmark smoke	1.971	0	Timing/state path completed

Table 10: Saturated reference-backend timing. Random token generation is outside timed regions. Values are descriptive for this machine and code path, not baseline comparisons.

Metric	Minimum	Median	Maximum
Decode throughput (tokens/s)	127.96	143.71	180.28
Decode latency (ms/token)	—	6.9586	—
Prefill throughput (tokens/s)	500.30	587.36	916.27

References

- Simran Arora, Sabri Eyuboglu, Aman Timalsina, Isys Johnson, Michael Poli, James Zou, Atri Rudra, and Christopher Ré. Zoology: Measuring and improving recall in efficient language models. In *The Twelfth International Conference on Learning Representations*, 2024a. URL <https://openreview.net/forum?id=LY3ukUANko>.
- Simran Arora, Sabri Eyuboglu, Michael Zhang, Aman Timalsina, Silas Alberti, James Zou, Atri Rudra, and Christopher Re. Simple linear attention language models balance the recall-throughput tradeoff. In *Proceedings of the 41st International Conference on Machine Learning*, volume 235 of *Proceedings of Machine Learning Research*, pages 1763–1840. PMLR, 2024b. URL <https://proceedings.mlr.press/v235/arora24a.html>.
- Ali Behrouz, Zeman Li, Yuan Deng, Peilin Zhong, Meisam Razaviyayn, and Vahab Mirrokni. Memory caching: RNNs with growing memory, 2026. URL <https://arxiv.org/abs/2602.24281>. Preprint.
- Satwik Bhattamishra, Michael Hahn, Phil Blunsom, and Varun Kanade. Separations in the representational capabilities of transformers and recurrent architectures. In *Advances in Neural Information Processing Systems*, volume 37, 2024. doi: 10.52202/079017-1135. URL https://proceedings.neurips.cc/paper_files/paper/2024/hash/3f630b20b7b3ac76d3a0016fe29b6dc0-Abstract-Conference.html.
- Matthew Brand. Fast low-rank modifications of the thin singular value decomposition. *Linear Algebra and its Applications*, 415(1):20–30, 2006. doi: 10.1016/j.laa.2005.07.021. URL <https://doi.org/10.1016/j.laa.2005.07.021>.
- Loïc Cabannes, Pierre-Emmanuel Mazaré, Gergely Szilvasy, Matthijs Douze, Maria Lomeli, Ilze Amanda Auzina, Justin Carpentier, Gabriel Synnaeve, and Hervé Jégou. Sparse delta memory: Scaling the state of linear RNNs through sparsity, 2026. URL <https://arxiv.org/abs/2607.07386>. Preprint.
- Wanyun Cui. A hippocampus for linear attention: An exact memory for what the recurrent state forgets, 2026. URL <https://arxiv.org/abs/2607.02303>. Preprint.
- Tri Dao. FlashAttention-2: Faster attention with better parallelism and work partitioning. In *The Twelfth International Conference on Learning Representations*, 2024. URL https://proceedings.iclr.cc/paper_files/paper/2024/hash/98ed250b203d1ac6b24bbcf263e3d4a7-Abstract-Conference.html.
- Tri Dao and Albert Gu. Transformers are SSMs: Generalized models and efficient algorithms through structured state space duality. In *Proceedings of the 41st International Conference on Machine Learning*, volume 235 of *Proceedings of Machine Learning Research*, pages 10041–10071. PMLR, 2024. URL <https://proceedings.mlr.press/v235/dao24a.html>.
- Tri Dao, Daniel Y. Fu, Stefano Ermon, Atri Rudra, and Christopher Ré. FlashAttention: Fast and memory-efficient exact attention with IO-awareness. In *Advances in Neural Information Processing Systems*, volume 35, pages 16344–16359, 2022. doi: 10.52202/068431-1189. URL <https://proceedings.neurips.cc/paper/2022/hash/67d57c32e20fd0a7a302cb81d36e40d5-Abstract-Conference.html>.
- Jusen Du, Jiayi Hu, Zhang Tao, Weigao Sun, and Yu Cheng. Native hybrid attention for efficient sequence modeling. In *Proceedings of the 64th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 3826–3842, San Diego, California, United States, 2026. Association for Computational Linguistics. doi: 10.18653/v1/2026.acl-long.176. URL <https://aclanthology.org/2026.acl-long.176/>.

- Carl Eckart and Gale Young. The approximation of one matrix by another of lower rank. *Psychometrika*, 1(3):211–218, 1936. doi: 10.1007/BF02288367. URL <https://doi.org/10.1007/BF02288367>.
- Mina Ghashami, Edo Liberty, Jeff M. Phillips, and David P. Woodruff. Frequent directions: Simple and deterministic matrix sketching. *SIAM Journal on Computing*, 45(5):1762–1792, 2016. doi: 10.1137/15M1009718. URL <https://doi.org/10.1137/15M1009718>.
- Albert Gu and Tri Dao. Mamba: Linear-time sequence modeling with selective state spaces. In *First Conference on Language Modeling*, 2024. URL <https://openreview.net/forum?id=tEYskw1VY2>. COLM 2024 Outstanding Paper.
- Ali Hatamizadeh, Yejin Choi, and Jan Kautz. Gated DeltaNet-2: Decoupling erase and write in linear attention, 2026. URL <https://arxiv.org/abs/2605.22791>. Preprint; Gated DeltaNet-2 technical report.
- Alston S. Householder. Unitary triangularization of a nonsymmetric matrix. *Journal of the ACM*, 5(4):339–342, 1958. doi: 10.1145/320941.320947. URL <https://doi.org/10.1145/320941.320947>.
- Cheng-Ping Hsieh, Simeng Sun, Samuel Kriman, Shantanu Acharya, Dima Rekesh, Fei Jia, Yang Zhang, and Boris Ginsburg. RULER: What’s the real context size of your long-context language models? In *First Conference on Language Modeling*, 2024. URL <https://openreview.net/forum?id=kIoBbc76Sy>.
- Praneeth Kacham, Vahab Mirrokni, and Peilin Zhong. PolySketchFormer: Fast transformers via sketching polynomial kernels. In *Proceedings of the 41st International Conference on Machine Learning*, volume 235 of *Proceedings of Machine Learning Research*, pages 22748–22770. PMLR, 2024. URL <https://proceedings.mlr.press/v235/kacham24a.html>.
- Angelos Katharopoulos, Apoorv Vyas, Nikolaos Pappas, and François Fleuret. Transformers are RNNs: Fast autoregressive transformers with linear attention. In *Proceedings of the 37th International Conference on Machine Learning*, volume 119 of *Proceedings of Machine Learning Research*, pages 5156–5165. PMLR, 2020. URL <https://proceedings.mlr.press/v119/katharopoulos20a.html>.
- Kimi Team. Kimi linear: An expressive, efficient attention architecture, 2025. URL <https://arxiv.org/abs/2510.26692>. Preprint; Kimi Linear technical report.
- Yuri Kuratov, Aydar Bulatov, Petr Anokhin, Ivan Rodkin, Dmitry Sorokin, Artyom Sorokin, and Mikhail Burtsev. BABILong: Testing the limits of LLMs with long context reasoning-in-a-haystack. In *Advances in Neural Information Processing Systems: Datasets and Benchmarks Track*, 2024. URL <https://arxiv.org/abs/2406.10149>.
- Aakash Lahoti, Kevin Y. Li, Berlin Chen, Caitlin Wang, Aviv Bick, J. Zico Kolter, Tri Dao, and Albert Gu. Mamba-3: Improved sequence modeling using state space principles. In *The Fourteenth International Conference on Learning Representations*, 2026. URL <https://iclr.cc/virtual/2026/poster/10010352>. ICLR 2026 Oral.
- Ali Modarressi, Hanieh Deilamsalehy, Franck Dernoncourt, Trung Bui, Ryan A. Rossi, Seunghyun Yoon, and Hinrich Schütze. NoLiMa: Long-context evaluation beyond literal matching. In *Proceedings of the 42nd International Conference on Machine Learning*, 2025. URL <https://arxiv.org/abs/2502.05167>. Accepted at ICML 2025.
- Tsendsuren Munkhdalai, Manaal Faruqui, and Siddharth Gopal. Leave no context behind: Efficient infinite context transformers with infini-attention, 2024. URL <https://arxiv.org/abs/2404.07143>. Preprint.

- Ninh Pham and Rasmus Pagh. Fast and scalable polynomial kernels via explicit feature maps. In *Proceedings of the 19th ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, pages 239–247. Association for Computing Machinery, 2013. doi: 10.1145/2487575.2487591. URL <https://doi.org/10.1145/2487575.2487591>.
- Ninh Pham and Rasmus Pagh. Tensor sketch: Fast and scalable polynomial kernel approximation, 2025. URL <https://arxiv.org/abs/2505.08146>. Preprint; extension of the KDD 2013 paper and correction of its variance bound.
- Imanol Schlag, Kazuki Irie, and Jürgen Schmidhuber. Linear transformers are secretly fast weight programmers. In *Proceedings of the 38th International Conference on Machine Learning*, volume 139 of *Proceedings of Machine Learning Research*, pages 9355–9366. PMLR, 2021. URL <https://proceedings.mlr.press/v139/schlag21a.html>.
- Songlin Yang, Bailin Wang, Yu Zhang, Yikang Shen, and Yoon Kim. Parallelizing linear transformers with the delta rule over sequence length. In *Advances in Neural Information Processing Systems*, volume 37, 2024. doi: 10.52202/079017-3668. URL https://proceedings.neurips.cc/paper_files/paper/2024/hash/d13a3eae72366e61dfdc7eea82eeb685-Abstract-Conference.html.
- Songlin Yang, Jan Kautz, and Ali Hatamizadeh. Gated delta networks: Improving Mamba2 with delta rule. In *The Thirteenth International Conference on Learning Representations*, 2025. URL <https://openreview.net/forum?id=r8H7xhYPwz>.
- Howard Yen, Tianyu Gao, Minmin Hou, Ke Ding, Daniel Fleischer, Peter Izsak, Moshe Wasserblat, and Danqi Chen. HELMET: How to evaluate long-context language models effectively and thoroughly. In *The Thirteenth International Conference on Learning Representations*, 2025. URL https://proceedings.iclr.cc/paper_files/paper/2025/hash/f5332c8273d02729730a9c24dec2135e-Abstract-Conference.html.